

Leren & Redeneren

Graphical Models

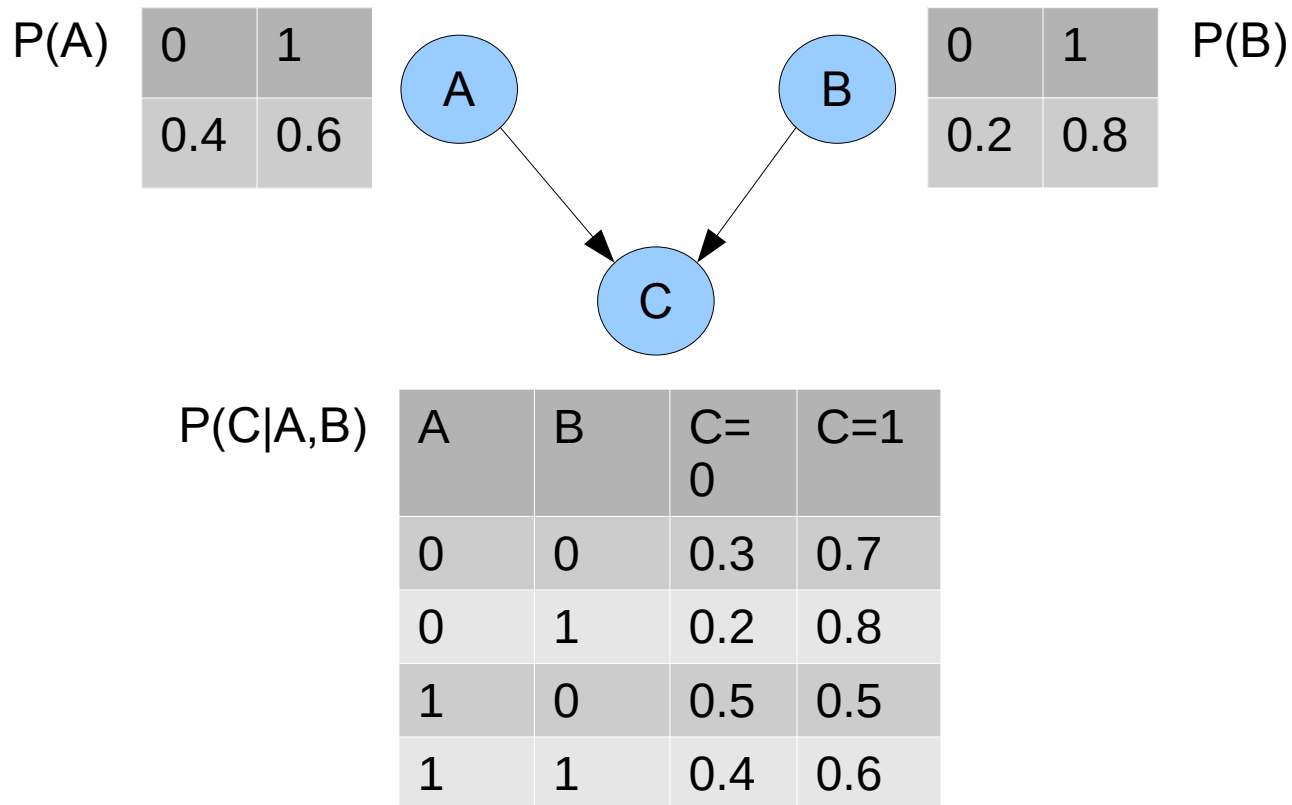
Siegfried Nijssen

Probabilistic Graphical Models

- *Annotated graphs* representing distributions over variables
- Two well-known types of probabilistic graphical models:
 - ◆ Bayesian networks
 - ◆ Markov networks

Bayesian Networks

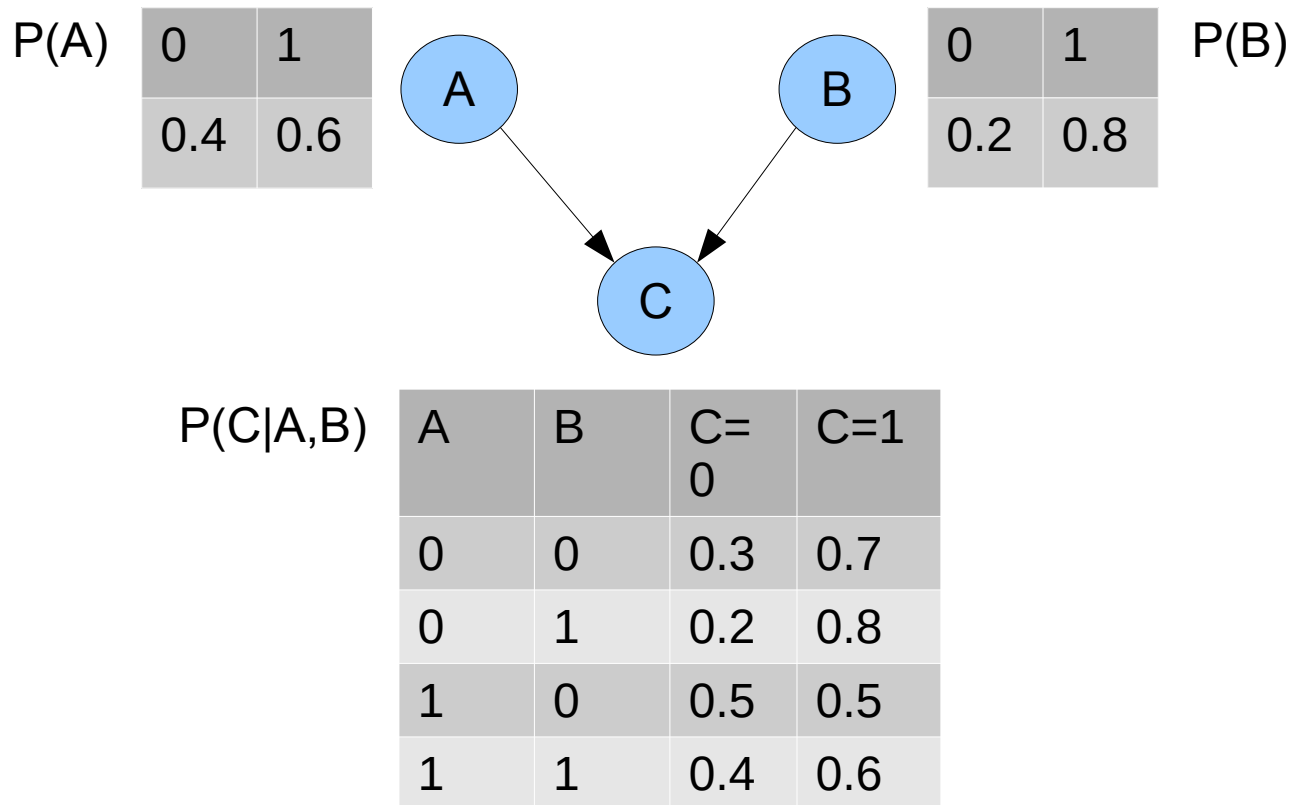
- Example Bayesian network over Boolean variables:



$$P(A=1, B=0, C=1) = P(A=1) * P(B=0) * P(C=1|A=1, B=0) = 0.6 * 0.2 * 0.5 = 0.06$$

Bayesian Networks

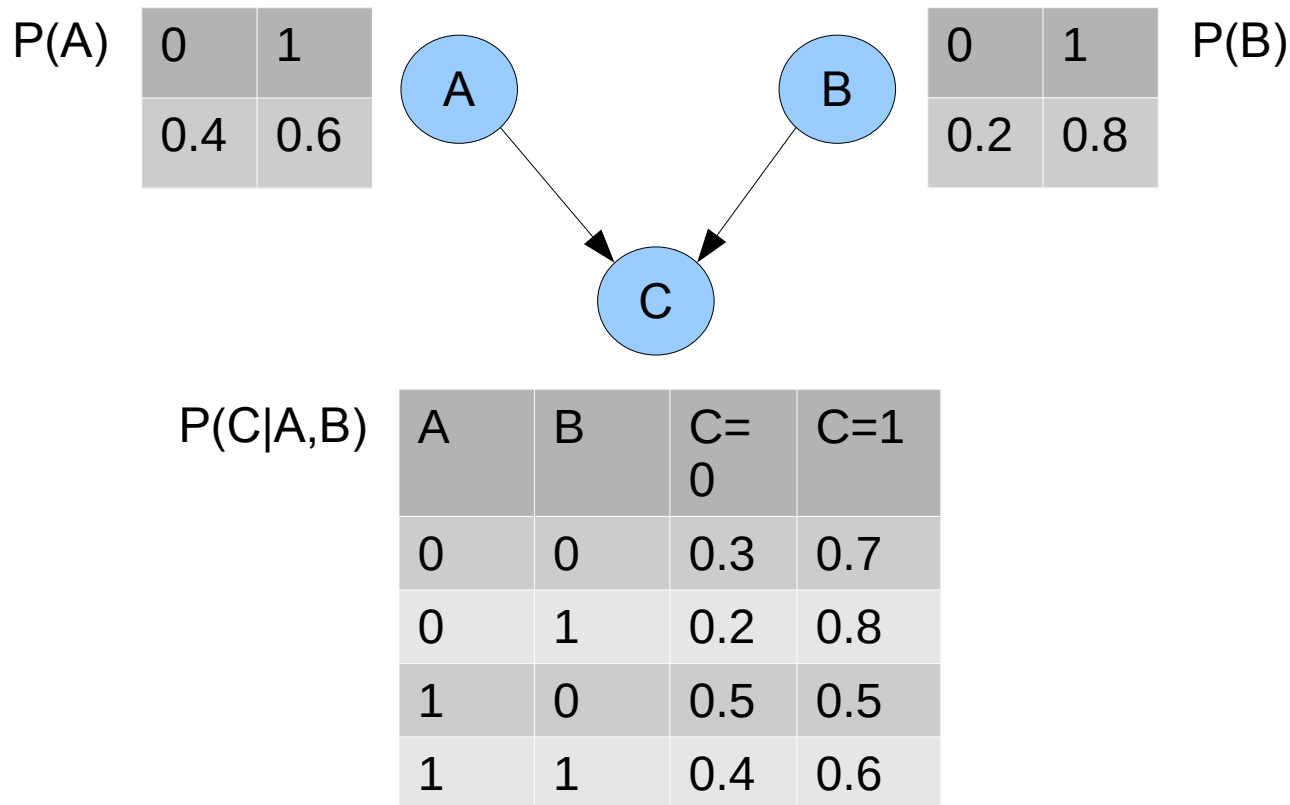
- Example Bayesian network over Boolean variables:



- The network expresses conditional dependencies: a variable depends only on the value(s) of its parent(s)

Bayesian Networks

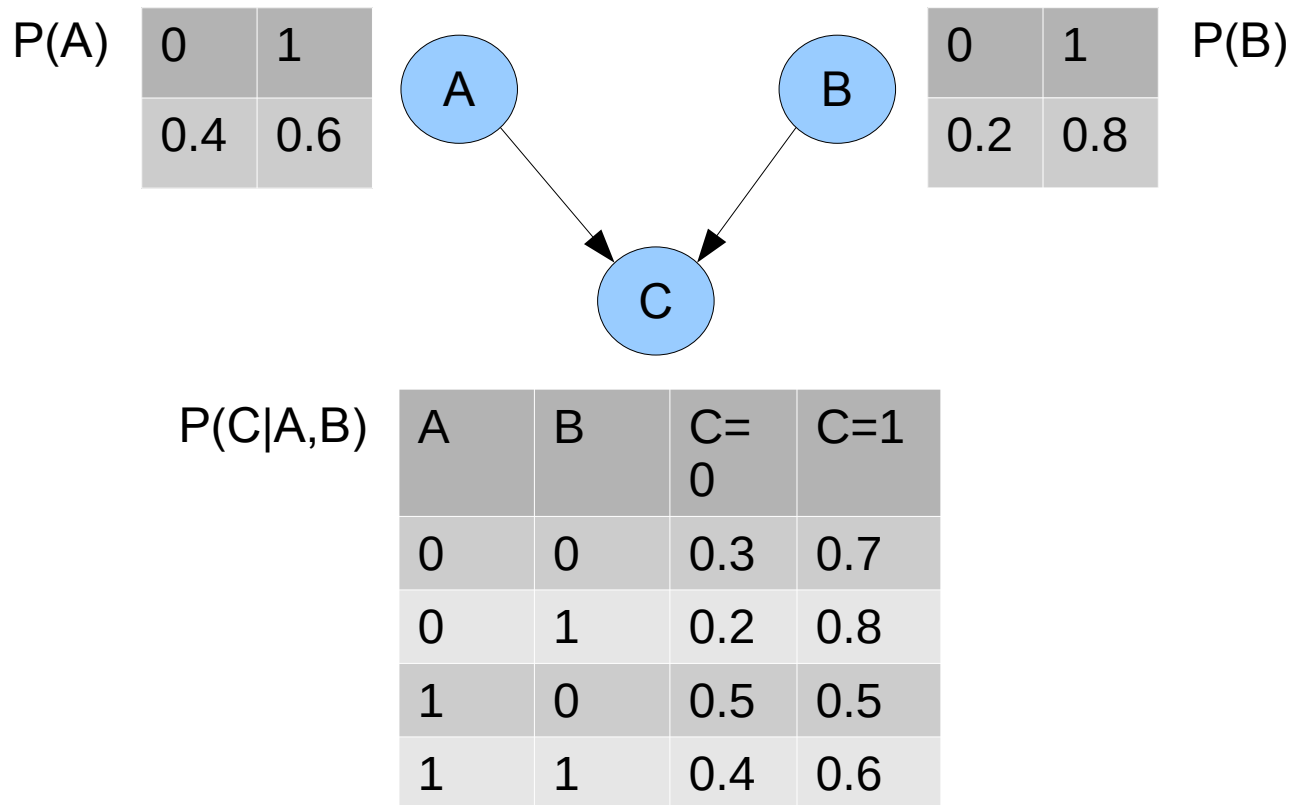
- Example Bayesian network over Boolean variables:



- $$P(X_1=v_1, X_2=v_2, \dots, X_n=v_n) = \prod_{\text{all variables } i} P(X_i=v_i | \text{parents}(X_i))$$

Bayesian Networks

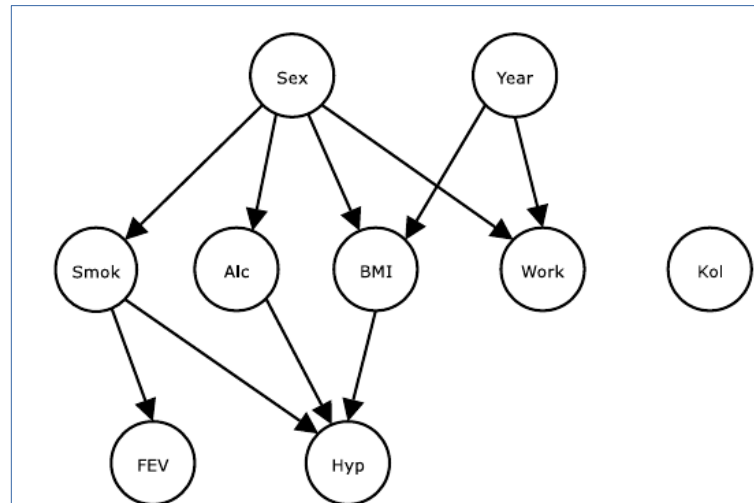
- What is the probability of $P(C=1)$?



$$P(C=1) = P(C=1, A=0, B=0) + P(C=1, A=0, B=1) + P(C=1, A=1, B=0) + P(C=1, A=1, B=1)$$

Bayesian Networks: Structure

- The structure of a Bayesian Network consists of a *Directed Acyclic Graph (DAG)*

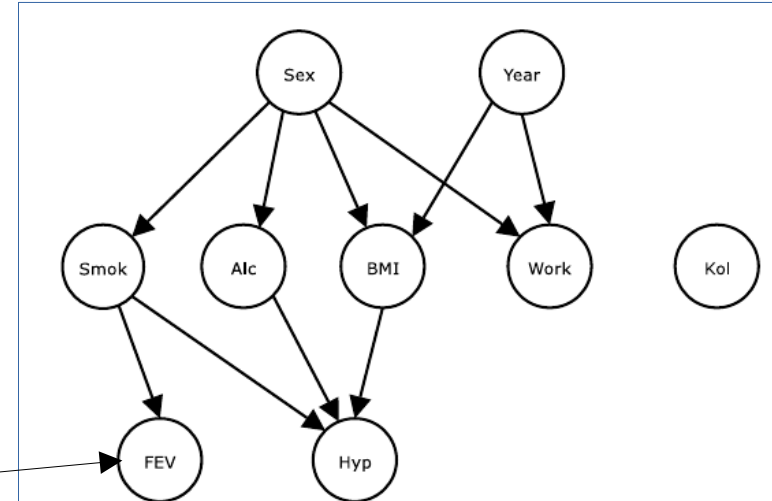


- Each variable v (Sex, Year, Smok, ...) corresponds to a node and has a domain $D(v)$ (Boolean, Categorical, Real, ...)

Bayesian Networks: Parameters

- For now we assume variables only have categorical values
- In this case for every node in the network we have a *conditional probability table (CPT)*

Smok	FEV	
0	0	0.3
0	1	0.7
1	0	0.8
1	1	0.2



Bayesian Networks:

Inferring Marginal Probabilities

- Given
 - ♦ a subset of *query* variables X ,
 - ♦ a vector $\vec{x}$ having for each of the variables in X a value,the *marginal probability* is the probability $P(X = \vec{x})$
 - ♦ e.g., if $X = (\text{Sex}, \text{Year})$ and $\vec{x} = (\text{M}, 1999)$, for the earlier network, $P(\text{Sex}=\text{M}, \text{Year}=1999)$ is a marginal probability
- We can calculate a marginal probability by summing over all assignments to all other variables, i.e.

$$P(X = \vec{x}) = \sum_{\vec{y} \in D(Y)} P(X = \vec{x} \wedge Y = \vec{y})$$

Here $D(Y) = D(y_1) \times \cdots \times D(y_n)$ and Y is the set of all variables not in the query variables X

- ♦ *However this naïve approach could be time consuming*

Bayesian Networks

- Traditionally seen as machine learning model
- Advantages:
 - ◆ Interpretable models, both in structure and in parameters if tables are used
 - ◆ Generative: one can generate data from them
 - ◆ Allow for providing evidence on any set of variables, and to query on any other set of variables
 - ◆ Well-known algorithms for learning parameters
- Disadvantages:
 - ◆ Inference can be costly (exponential)
 - ◆ Structure learning is challenging; not easy to find good models (e.g., as graphs need to be DAGs)

Constraint Programming

- General approach for solving constraint satisfaction and optimization problems (reasoning problems)
- Examples are scheduling and planning problems

MODEL (by user)
+ SEARCH (by solver)

A CP model

- variables

$E_{11} \dots E_{99}$

- domains

$E_{xy} : \{1 \dots 9\}$

- constraints

$\text{all_different}([E_{1x}]), \dots$

$\text{all_different}([E_{x1}]), \dots$

$\text{all_different}([E_{11} \dots E_{33}]), \dots$

	2	3				5		
			8	2		7	9	3
⋮	6	4			9	8		
E_{41}	E_{42}	⋮	2		7		4	
E_{43}	⋮	9		8		1		
⋮	4	2						
	8						3	
			6			2		1
4					1		8	

CP Search

Two key principles to make domains smaller:

- **Propagation** of constraints (*active constraints*)

eg. alldiff(X,Y,Z) $X=\{1\}, Y=\{1,2\}, Z=\{1,2,3,4\} \rightarrow Y=\{2\}, Z=\{3,4\}$

Every constraint is implemented by a propagator.

- **Branch** over values of variables

eg. Propagation at fixpoint $\rightarrow$ branch over $Z=\{3\}$

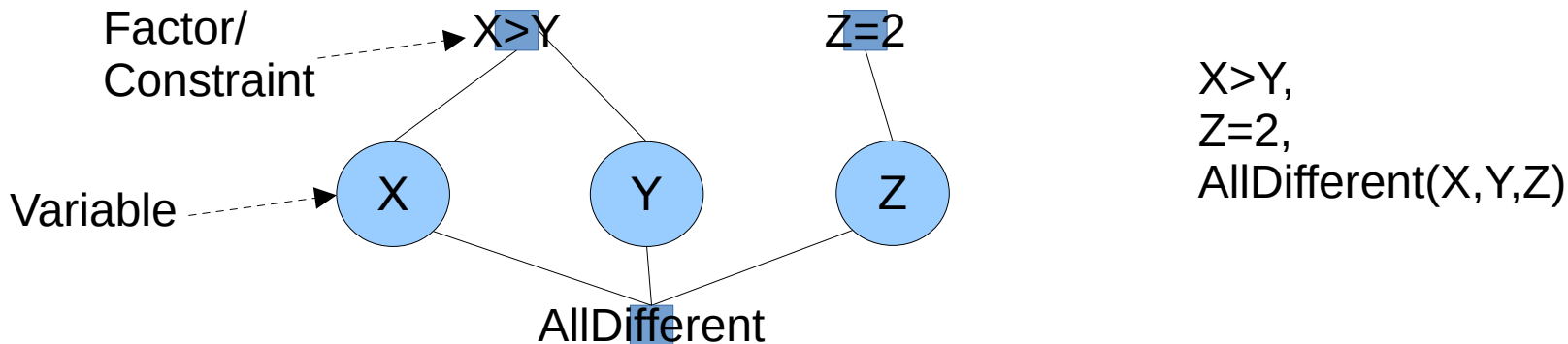
Search is recursive and complete

Factor Graphs

- General framework for problems modeled over variables
- Bayesian Networks and Constraint Programs can both be seen as instances of *factor graphs*
 - ◆ Hence *factor graphs* can be seen as generalizing machine learning problems and reasoning problems
- Factor graphs are bipartite graphs with
 - ◆ One set of nodes representing *variables*
 - ◆ The other set of nodes representing *factors* that link ≥ 1 variables
 - ◆ For an assignment to the variables in a factor, the factor defines a value
 - ◆ The value of an assignment to all variables is determined by the product of the values for all the factors

Factor Graphs: Constraint Programs

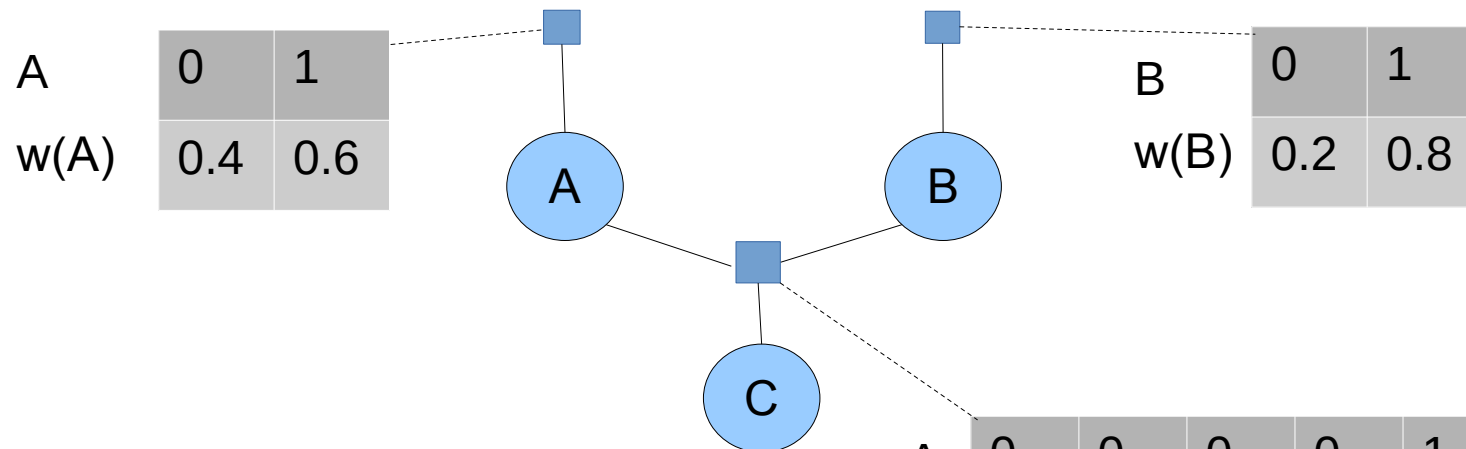
- Factors = Constraints
- A constraint (=factor) has weight of 1 for an assignment if it is true for that assignment, otherwise 0
- An assignment has weight 1 if all constraints have weight 1



X	Y	Z	Weight for $X > Y$	Weight for $Z = 2$	Weight for $\text{AllDifferent}(X, Y, Z)$	Total Weight
3	1	2	1	1	1	1
3	2	2	1	1	0	0

Factor Graphs: Bayesian Networks

- Factors = (Conditional) Probabilities
- A factor defines a (conditional) probability for the variables in the factor

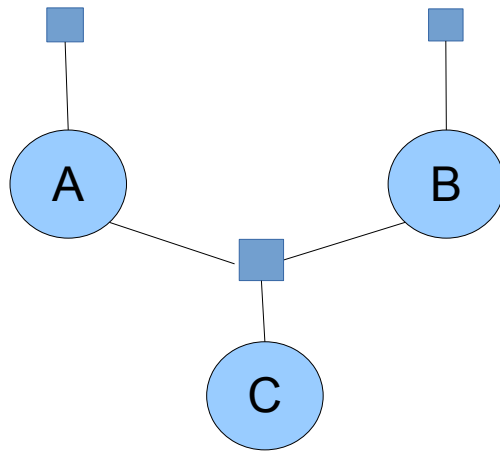


$$P(A=a, B=b, C=c) = w(A=a) * w(B=b) * w(A=a, B=b, C=c)$$

Here the table for a factor defines a weight for each possible assignment to the variables in the factor; weights have to correspond to a valid distribution

Factor Graphs: Markov Networks

- Factor = *Feature* with *weight*
- Feature: Set of *variable=value* pairs
- Weight: positive, negative, higher than 1, without further constraints



Feature	Weight
A=0	-0.91
A=1	-0.51
B=0	-1.61
B=1	-0.22
A=0,B=0,C=0	-1.20
A=0,B=0,C=1	-0.35
A=0,B=1,C=0	-1.61
A=0,B=1,C=1	-0.22
A=1,B=0,C=0	-0.69
A=1,B=0,C=1	-0.69
A=1,B=1,C=0	-0.92
A=1,B=1,C=1	-0.51

Markov Networks

Define the probability of an assignment to all variables as

$$P(I) = \frac{1}{Z} \exp \left(\sum_f w_f \cdot \text{match}(I_f, I) \right)$$

Diagram illustrating the components of the Markov Network probability formula:

- $\frac{1}{Z}$: Normalisation constant such that the sum over all assignments to all variables sums up to 1
- w_f : The weight of *feature* f
- $\text{match}(I_f, I)$: Is the feature compatible with the assignment to all variables
- I_f : The assignment in the feature

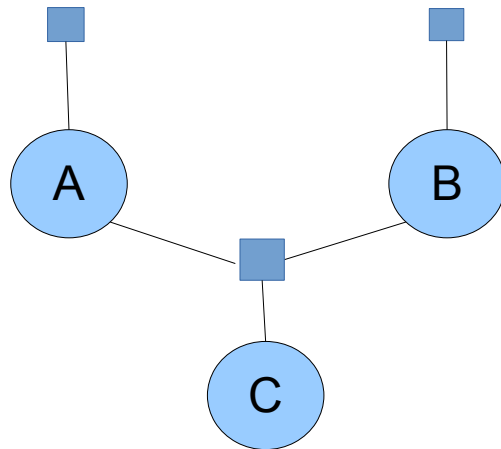
Normalisation constant such that the sum over all assignments to all variables sums up to 1

Markov Networks

Annoying normalization constant

$$P(A=0, B=0, C=0) =$$

$$\frac{\exp(w(A=0) + w(B=0) + w(A=0, B=0, C=0))}{Z} = \frac{\exp(-0.91 - 1.61 - 1.20)}{Z} = 0.024/Z$$



Feature	Weight
A=0	-0.91
A=1	-0.51
B=0	-1.61
B=1	-0.22
A=0, B=0, C=0	-1.20
A=0, B=0, C=1	-0.35
A=0, B=1, C=0	-1.61
A=0, B=1, C=1	-0.22
A=1, B=0, C=0	-0.69
A=1, B=0, C=1	-0.69
A=1, B=1, C=0	-0.92
A=1, B=1, C=1	-0.51

Markov Networks

It is not always necessary to know the normalization constant!

Example 1: marginal probability

$$P(A=0|B=0,C=0) =$$

$$P(A=0,B=0,C=0)/P(B=0,C=0) =$$

$$P(A=0,B=0,C=0)/(P(A=0,B=0,C=0)+P(A=1,B=0,C=0))=$$
$$w_1/(w_1+w_2) =$$

$$0.24/(0.24+0.06) =$$
$$0.8$$

where

$$w_1 = \exp(w(A=0,B=0,C=0)+w(A=0)+w(B=0)) = 0.24$$

$$w_2 = \exp(w(A=1,B=0,C=0)+w(A=1)+w(B=0)) = 0.06$$

Feature	Weight
A=0	-0.91
A=1	-0.51
B=0	-1.61
B=1	-0.22
A=0,B=0,C=0	-1.20
A=0,B=0,C=1	-0.35
A=0,B=1,C=0	-1.61
A=0,B=1,C=1	-0.22
A=1,B=0,C=0	-0.69
A=1,B=0,C=1	-0.69
A=1,B=1,C=0	-0.92
A=1,B=1,C=1	-0.51

Markov Networks

It is not always necessary to know the normalization constant!

Example 2: most probable assignment

$$\operatorname{argmax}_v P(A=v | B=0, C=0) =$$

$$\operatorname{argmax}_v P(A=v, B=0, C=0) / P(B=0, C=0) =$$

$$\operatorname{argmax}_v P(A=v, B=0, C=0) =$$

$$\operatorname{argmax}_v \exp(w(A=v, B=0, C=0) + w(A=v) + w(B=0)) / \mathbf{Z} =$$

$$\operatorname{argmax}_v w(A=v, B=0, C=0) + w(A=v) =$$

1

Feature	Weight
A=0	-0.91
A=1	-0.51
B=0	-1.61
B=1	-0.22
A=0,B=0,C=0	-1.20
A=0,B=0,C=1	-0.35
A=0,B=1,C=0	-1.61
A=0,B=1,C=1	-0.22
A=1,B=0,C=0	-0.69
A=1,B=0,C=1	-0.69
A=1,B=1,C=0	-0.92
A=1,B=1,C=1	-0.51

Markov Networks

Calculating Z

Feature	Weight
A=0	1
A=1	3
B=0	4
B=1	1
A=0,B=0,C=0	5
C=1	1

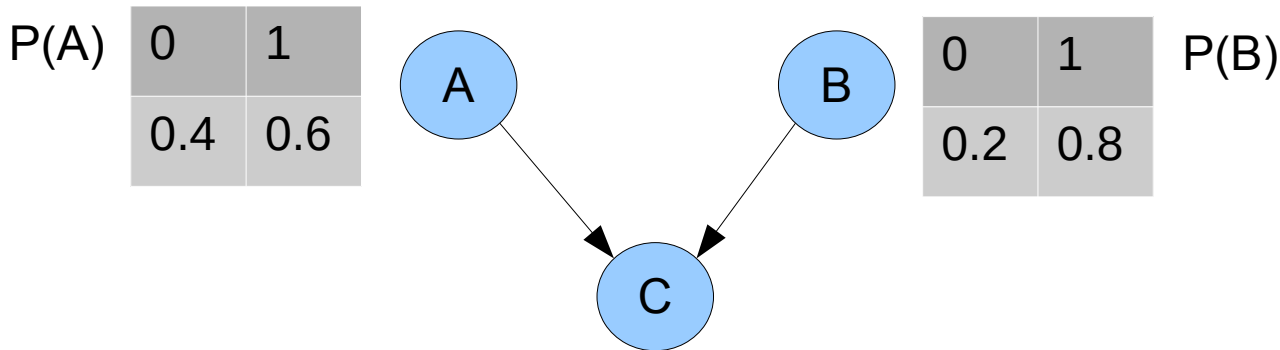
Assignment	Sum of weights	exp(sum)
A=0,B=0,C=0	1 + 4 + 5	22026
A=0,B=0,C=1	1 + 4 + 1	403
A=0,B=1,C=0	1 + 1	7
A=0,B=1,C=1	1 + 1 + 1	20
A=1,B=0,C=0	3 + 4	1096
A=1,B=0,C=1	3 + 4 + 1	2981
A=1,B=1,C=0	3 + 1	55
A=1,B=1,C=1	3 + 1 + 1	148
Z		26736

e.g. $P(A=0,B=0,C=0) = 22026/26736 = 0.824$

Larger weights $\rightarrow$ combinations that are given higher probabilities

Markov Networks

- Can always encode Bayesian Networks



$P(C|A,B)$

A	B	C=0	C=1
0	0	0.3	0.7
0	1	0.2	0.8
1	0	0.5	0.5
1	1	0.4	0.6

Feature	Weight
A=0	-0.91
A=1	-0.51
B=0	-1.61
B=1	-0.22
A=0,B=0,C=0	-1.20
A=0,B=0,C=1	-0.35
A=0,B=1,C=0	-1.61
A=0,B=1,C=1	-0.22
A=1,B=0,C=0	-0.69
A=1,B=0,C=1	-0.69
A=1,B=1,C=0	-0.92
A=1,B=1,C=1	-0.51

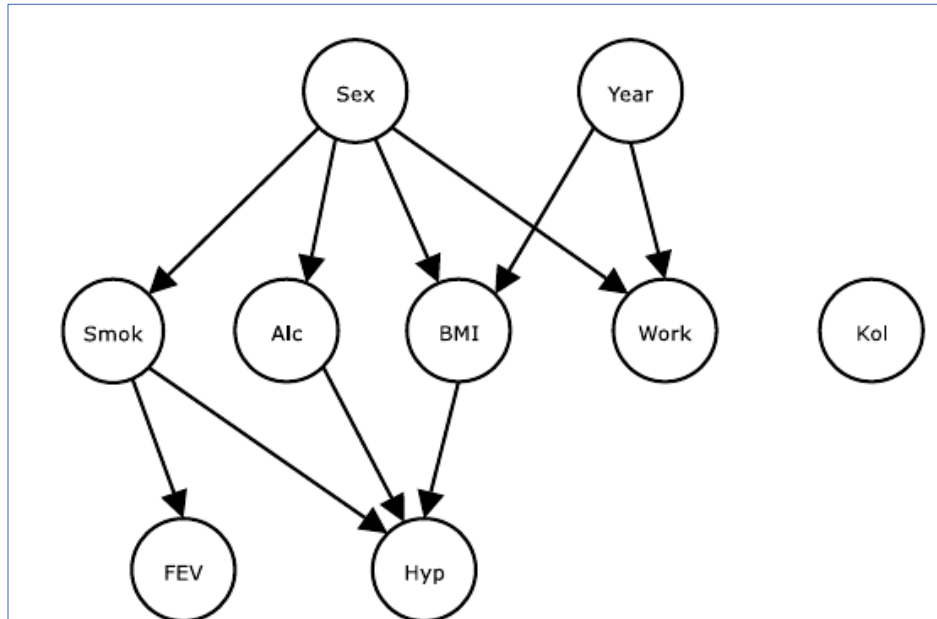
$\log(0.4)$

Markov Networks

- Advantages:
 - ◆ Graphs can have cycles
 - ◆ Weights can have any value
 - ◆ Generalize Bayesian Networks
- Disadvantages:
 - ◆ Z can be hard to determine
 - ◆ The factors can be harder to interpret

Factor Graphs: Marginals

- What is the probability of $P(\text{Hyp}=1)$?

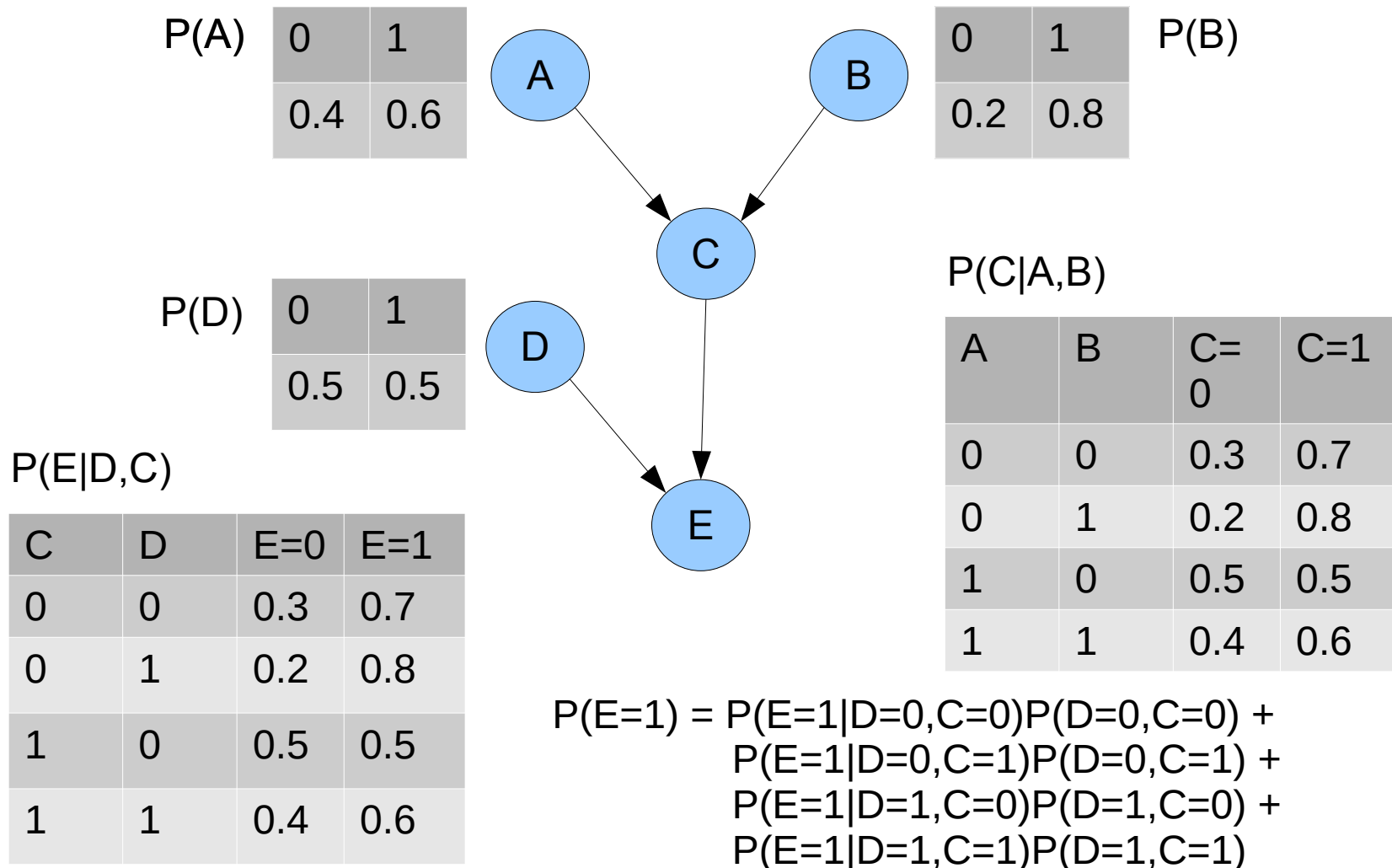


$P(\text{Hyp}=1, \text{Smok}=0, \text{Alc}=0, \text{BMI}=0, \text{Sex}=0, \text{Year}=0) +$
 $P(\text{Hyp}=1, \text{Smok}=0, \text{Alc}=0, \text{BMI}=0, \text{Sex}=0, \text{Year}=1) +$
 $P(\text{Hyp}=1, \text{Smok}=0, \text{Alc}=0, \text{BMI}=0, \text{Sex}=1, \text{Year}=0) +$
 $P(\text{Hyp}=1, \text{Smok}=0, \text{Alc}=0, \text{BMI}=0, \text{Sex}=1, \text{Year}=1) +$
 $P(\text{Hyp}=1, \text{Smok}=0, \text{Alc}=0, \text{BMI}=1, \text{Sex}=0, \text{Year}=0) +$
...

- In the worst case we would have 2^5 possible assignments
- In general the problem is #P complete (equivalent to counting the number of solutions to an NP complete problem)

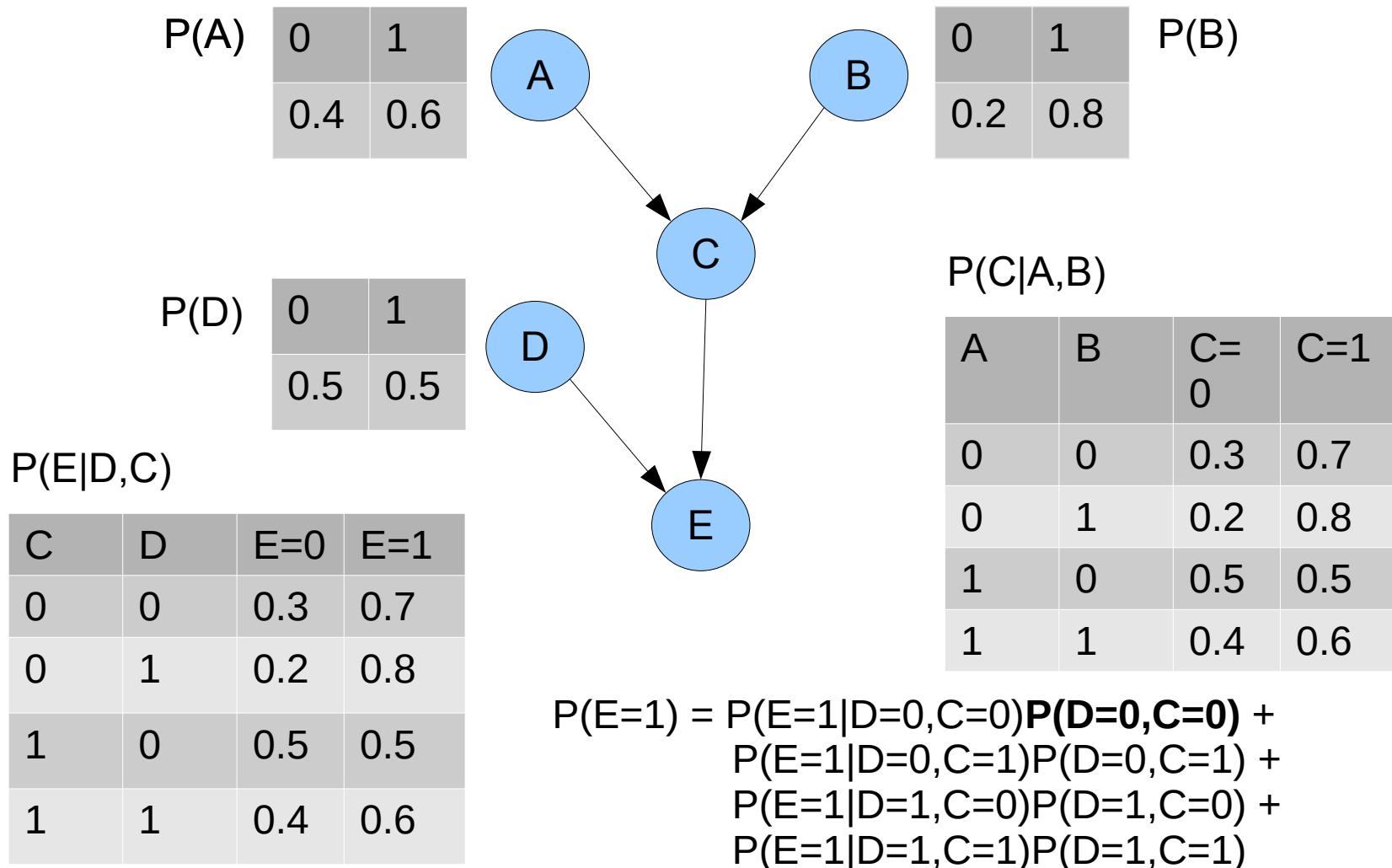
Bayesian Networks: Marginals

- Inference is simpler for some models. Example: BN trees



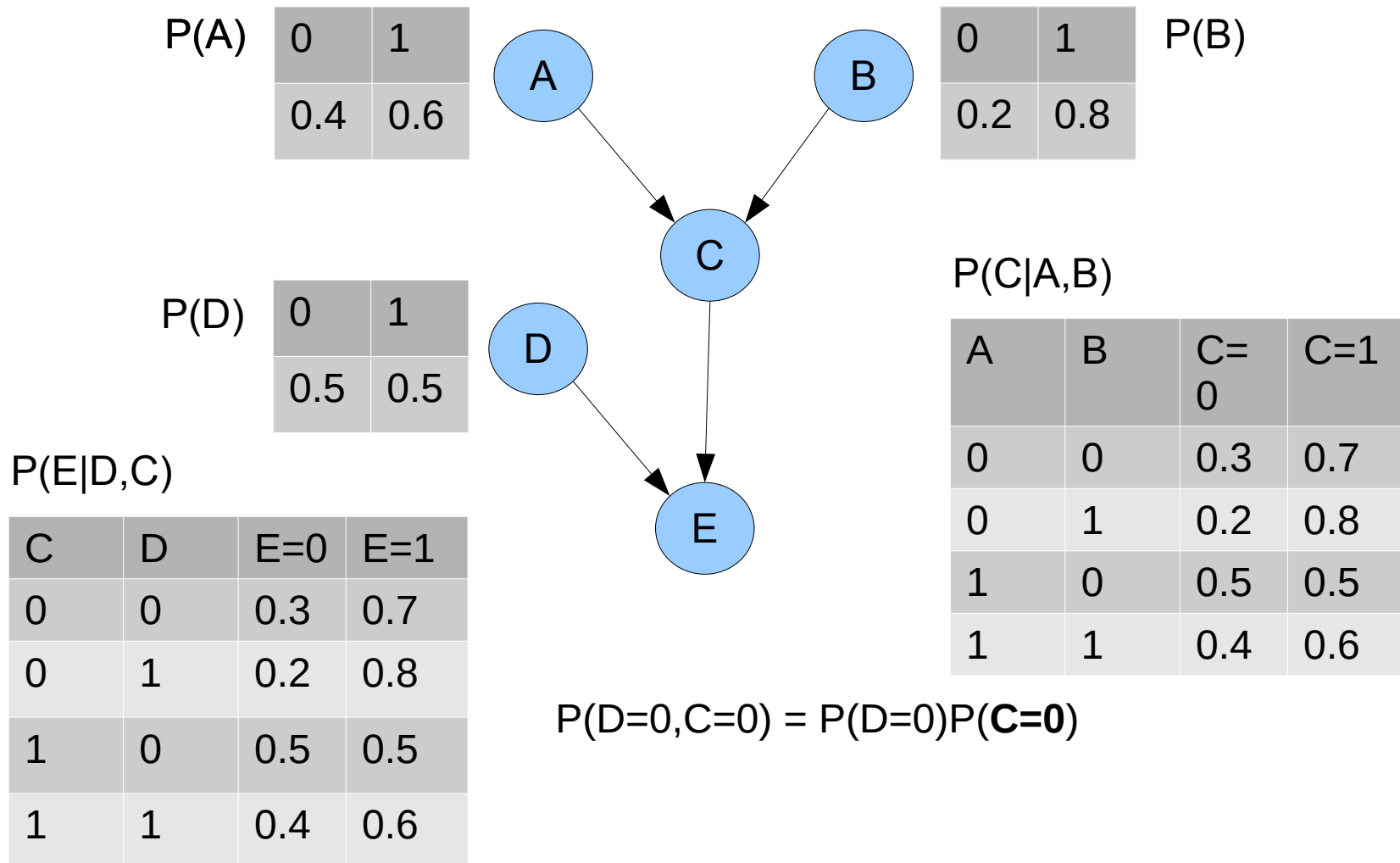
Bayesian Networks: Marginals

- Inference is simpler for some models. Example: BN trees



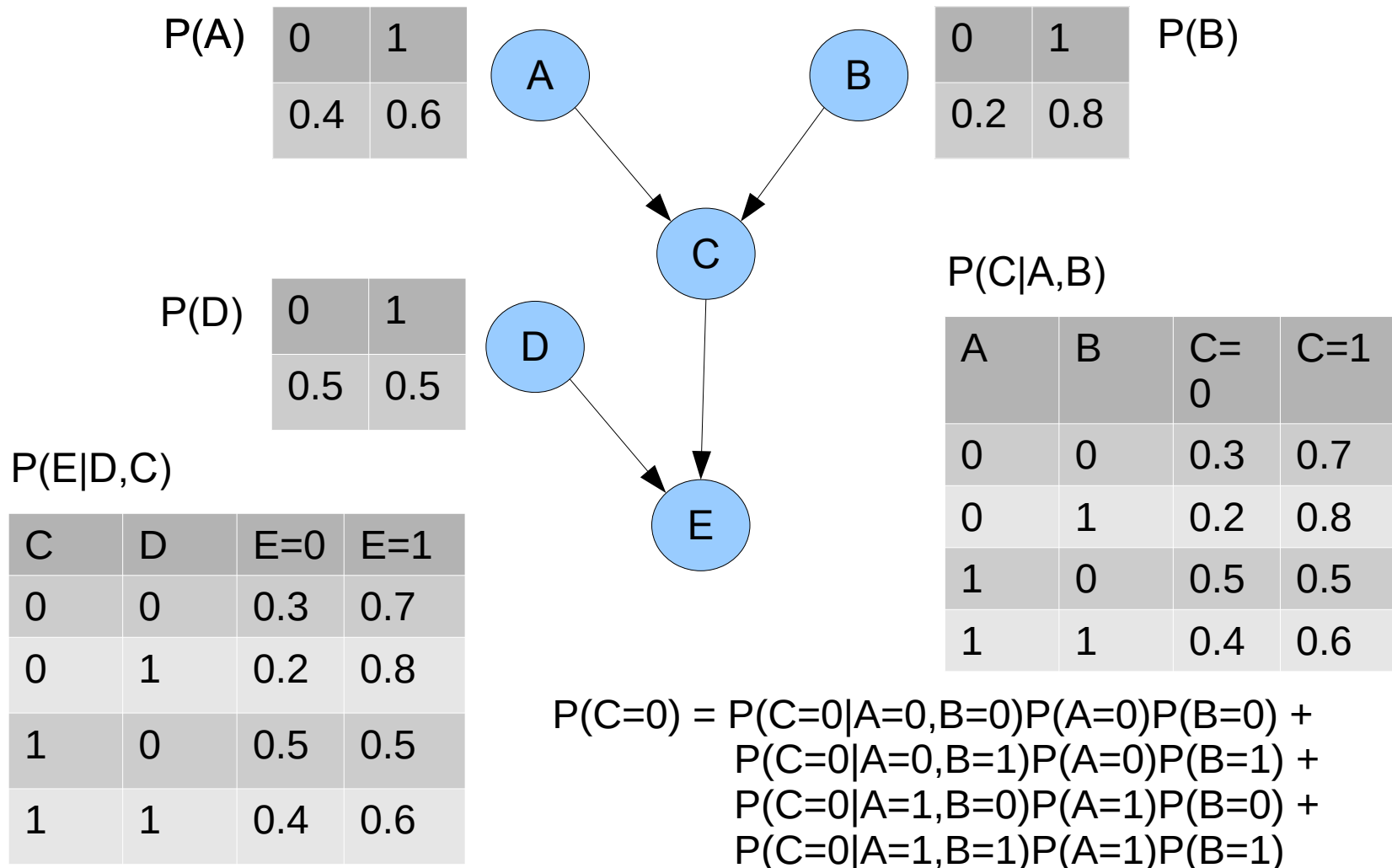
Bayesian Networks: Marginals

- Inference is simpler for some models. Example: BN trees



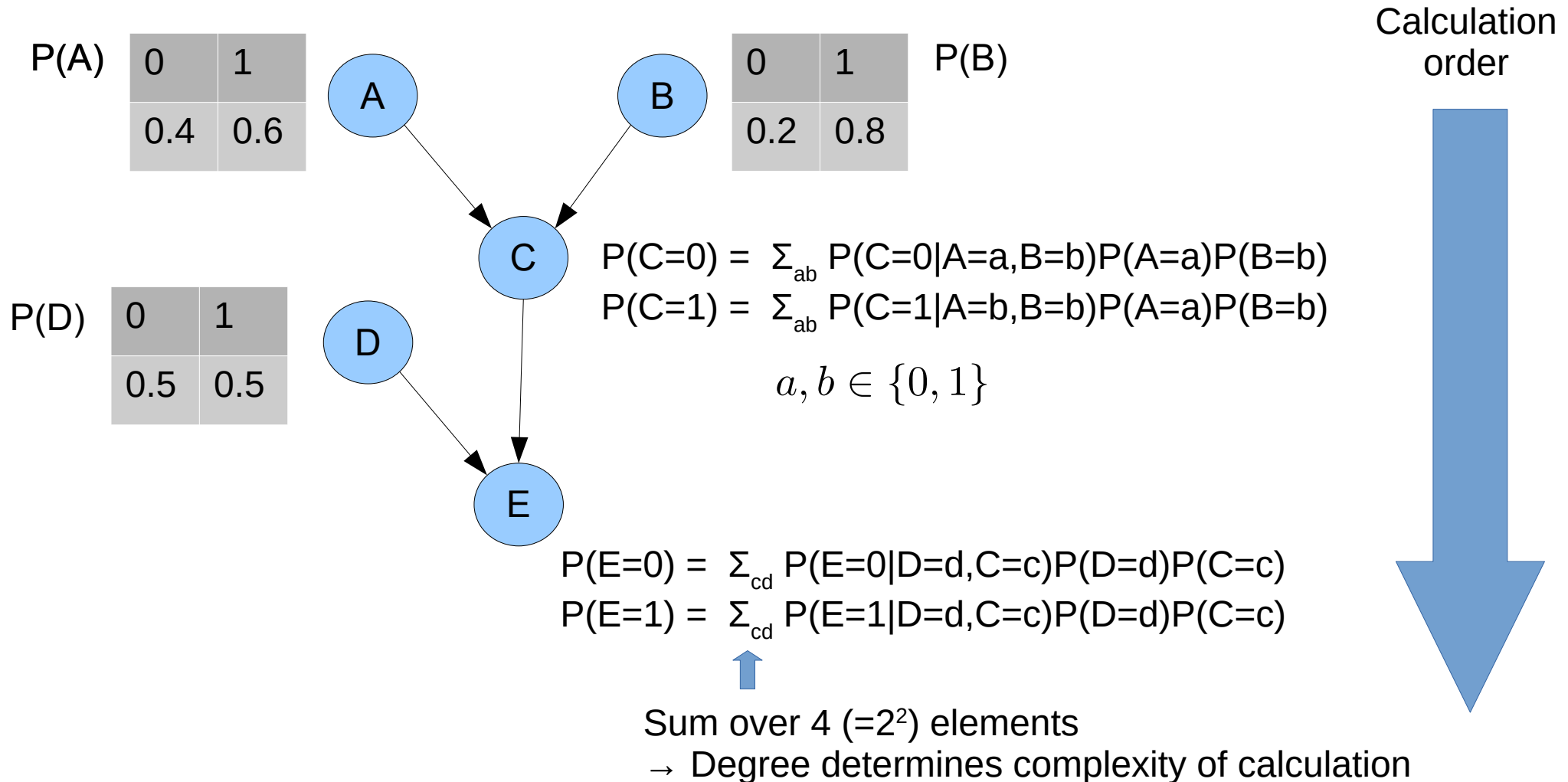
Bayesian Networks: Marginals

- Inference is simpler for some models. Example: BN trees



Bayesian Networks: Marginals

- Inference is simpler for some models. Example: BN trees



Message Passing on Factor Graphs

Let's rewrite the previous calculation as message passing on a factor graph

$$\mu_{1 \rightarrow A}(a) = w(A = a)$$

$$\mu_{2 \rightarrow B}(b) = w(B = b)$$

$$\mu_{A \rightarrow 3}(a) = \mu_{1 \rightarrow A}(a)$$

$$\mu_{B \rightarrow 3}(b) = \mu_{3 \rightarrow B}(b)$$

$$\mu_{4 \rightarrow D}(d) = w(D = d)$$

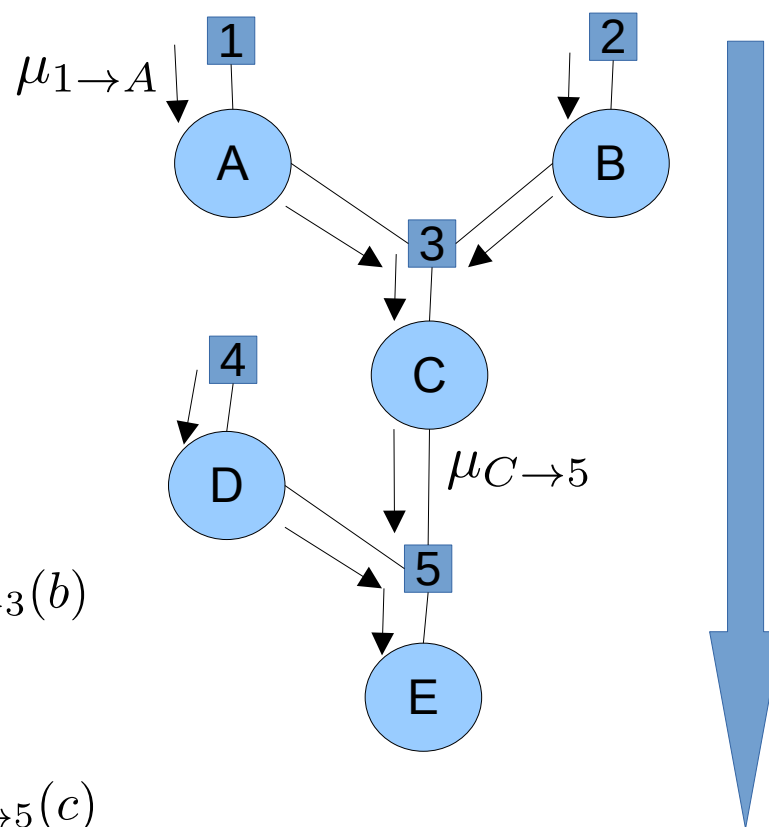
$$\mu_{D \rightarrow 5}(d) = \mu_{4 \rightarrow D}(d)$$

$$\mu_{3 \rightarrow C}(c) = \sum_{ab} w(A = a, B = b, C = c) \mu_{A \rightarrow 3}(a) \mu_{B \rightarrow 3}(b)$$

$$\mu_{C \rightarrow 5}(c) = \mu_{3 \rightarrow C}(c)$$

$$\mu_{5 \rightarrow E}(e) = \sum_{cd} w(C = c, D = d, E = e) \mu_{D \rightarrow 5}(d) \mu_{C \rightarrow 5}(c)$$

$$P(E = e) = \mu_{5 \rightarrow E}(e) / \left(\sum_{e'} \mu_{5 \rightarrow E}(e') \right)$$



Message Passing on Factor Graphs

Let's generalize message passing:

$$\mu_{factor \rightarrow variable}(v) = w(variable = v) \quad \text{if factor has one neighbour variable}$$

$$\mu_{1 \rightarrow A}(a) = w(A = a)$$

$$\mu_{2 \rightarrow B}(b) = w(B = b)$$

$$\mu_{A \rightarrow 3}(a) = \mu_{1 \rightarrow A}(a)$$

$$\mu_{B \rightarrow 3}(b) = \mu_{3 \rightarrow B}(b)$$

$$\mu_{4 \rightarrow D}(d) = w(D = d)$$

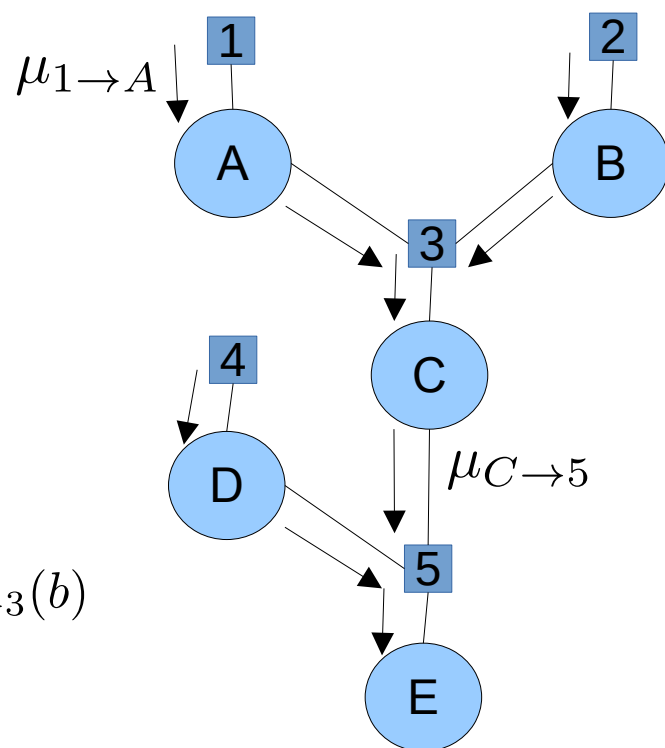
$$\mu_{D \rightarrow 5}(d) = \mu_{4 \rightarrow D}(d)$$

$$\mu_{3 \rightarrow C}(c) = \sum_{ab} w(A = a, B = b, C = c) \mu_{A \rightarrow 3}(a) \mu_{B \rightarrow 3}(b)$$

$$\mu_{C \rightarrow 5}(c) = \mu_{3 \rightarrow C}(c)$$

$$\mu_{5 \rightarrow E}(e) = \sum_{cd} w(C = c, D = d, E = e) \mu_{D \rightarrow 5}(d) \mu_{C \rightarrow 5}(c)$$

$$P(E = e) = \mu_{5 \rightarrow E}(e) / \left(\sum_{e'} \mu_{5 \rightarrow E}(e') \right)$$



Message Passing on Factor Graphs

Let's generalize message passing:

$$\mu_{factor \rightarrow var}(v) = \sum_{\bar{x} \in D(X)} w(var = v, X = \bar{x}) \prod_{var' \in X} \mu_{var' \rightarrow factor}(\bar{x}[var'])$$

$$\mu_{1 \rightarrow A}(a) = w(A = a)$$

$$\mu_{2 \rightarrow B}(b) = w(B = b)$$

$$\mu_{A \rightarrow 3}(a) = \mu_{1 \rightarrow A}(a)$$

$$\mu_{B \rightarrow 3}(b) = \mu_{3 \rightarrow B}(b)$$

$$\mu_{4 \rightarrow D}(d) = w(D = d)$$

$$\mu_{D \rightarrow 5}(d) = \mu_{4 \rightarrow D}(d)$$

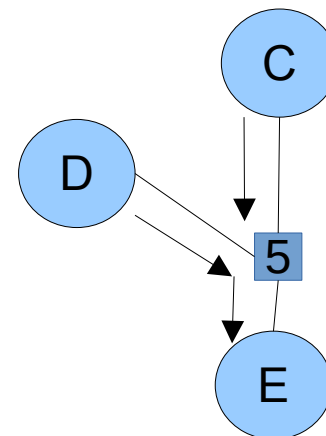
if $|X| \geq 1$, where X is the set of other variables neighboring *factor*

$$\mu_{3 \rightarrow C}(c) = \sum_{ab} w(A = a, B = b, C = c) \mu_{A \rightarrow 3}(a) \mu_{B \rightarrow 3}(b)$$

$$\mu_{C \rightarrow 5}(c) = \mu_{3 \rightarrow C}(c)$$

$$\mu_{5 \rightarrow E}(e) = \sum_{cd} w(C = c, D = d, E = e) \mu_{D \rightarrow 5}(d) \mu_{C \rightarrow 5}(c)$$

$$P(E = e) = \mu_{5 \rightarrow E}(e) / \left(\sum_{e'} \mu_{5 \rightarrow E}(e') \right)$$



Message Passing on Factor Graphs

Let's generalize message passing:

$$\mu_{variable \rightarrow factor}(v) = \prod_{f \in F} \mu_{f \rightarrow variable}(v)$$

$$\mu_{1 \rightarrow A}(a) = w(A = a)$$

$$\mu_{2 \rightarrow B}(b) = w(B = b)$$

$$\mu_{A \rightarrow 3}(a) = \mu_{1 \rightarrow A}(a)$$

$$\mu_{B \rightarrow 3}(b) = \mu_{3 \rightarrow B}(b)$$

$$\mu_{4 \rightarrow D}(d) = w(D = d)$$

$$\mu_{D \rightarrow 5}(d) = \mu_{4 \rightarrow D}(d)$$

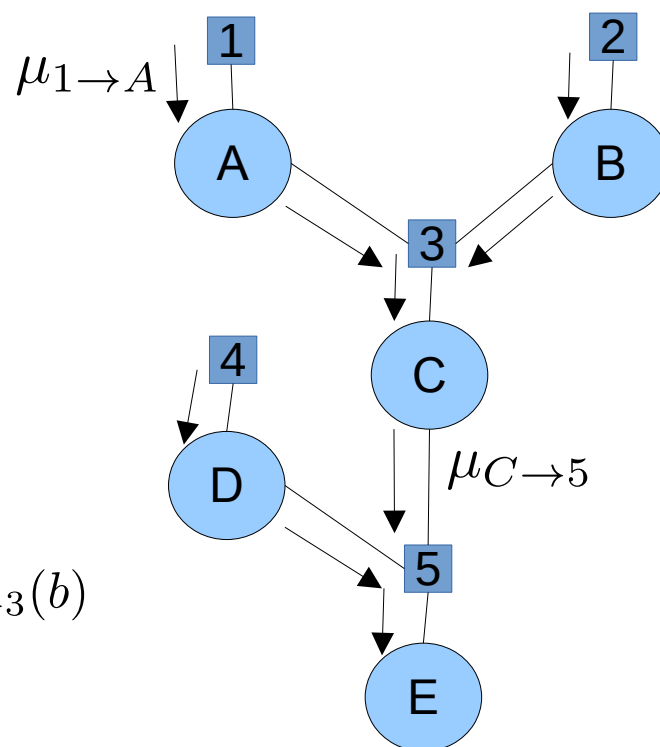
$$\mu_{3 \rightarrow C}(c) = \sum_{ab} w(A = a, B = b, C = c) \mu_{A \rightarrow 3}(a) \mu_{B \rightarrow 3}(b)$$

$$\mu_{C \rightarrow 5}(c) = \mu_{3 \rightarrow C}(c)$$

$$\mu_{5 \rightarrow E}(e) = \sum_{cd} w(C = c, D = d, E = e) \mu_{D \rightarrow 5}(d) \mu_{C \rightarrow 5}(c)$$

$$P(E = e) = \mu_{5 \rightarrow E}(e) / \left(\sum_{e'} \mu_{5 \rightarrow E}(e') \right)$$

where F is the set of other factors neighboring *variable*



Message Passing on Factor Graphs

Summary:

$$\mu_{factor \rightarrow variable}(v) = w(variable = v) \quad \text{if } factor \text{ has one neighbor } variable$$

$$\mu_{factor \rightarrow var}(v) = \sum_{\bar{x} \in D(X)} w(var = v, X = \bar{x}) \prod_{var' \in X} \mu_{var' \rightarrow factor}(\bar{x}[var'])$$

if $|X| \geq 1$, where X is the set of other variables neighboring $factor$

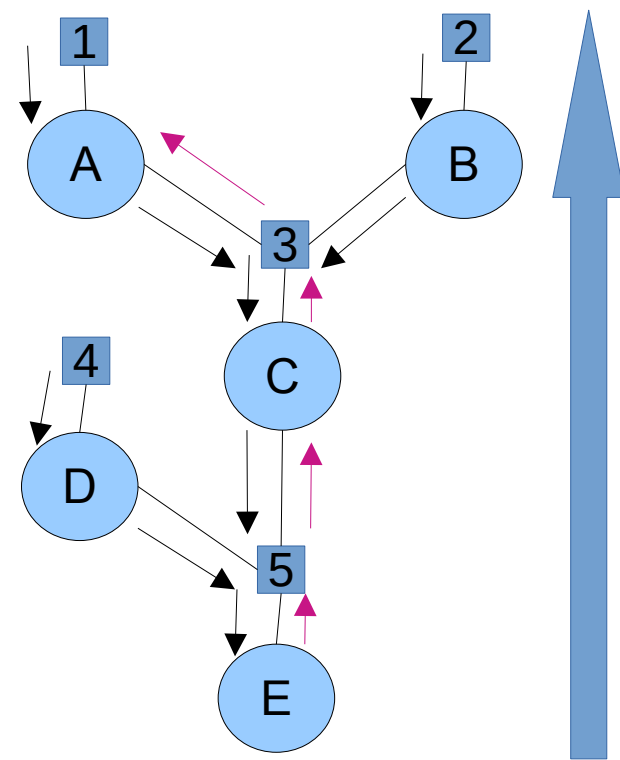
$$\mu_{variable \rightarrow factor}(v) = \prod_{f \in F} \mu_{f \rightarrow variable}(v) \quad \text{where } F \text{ is the set of other factors neighboring } variable$$

$$\mu_{variable \rightarrow factor}(v) = 1 \quad \text{if } variable \text{ has only one neighbor}$$

Message Passing on Factor Graphs

Calculating $P(A=a)$?

$$\rightarrow P(A = a) = \mu_{1 \rightarrow A}(a) \mu_{3 \rightarrow A}(a) / \left(\sum_{a'} \mu_{1 \rightarrow A}(a') \mu_{3 \rightarrow A}(a') \right)$$

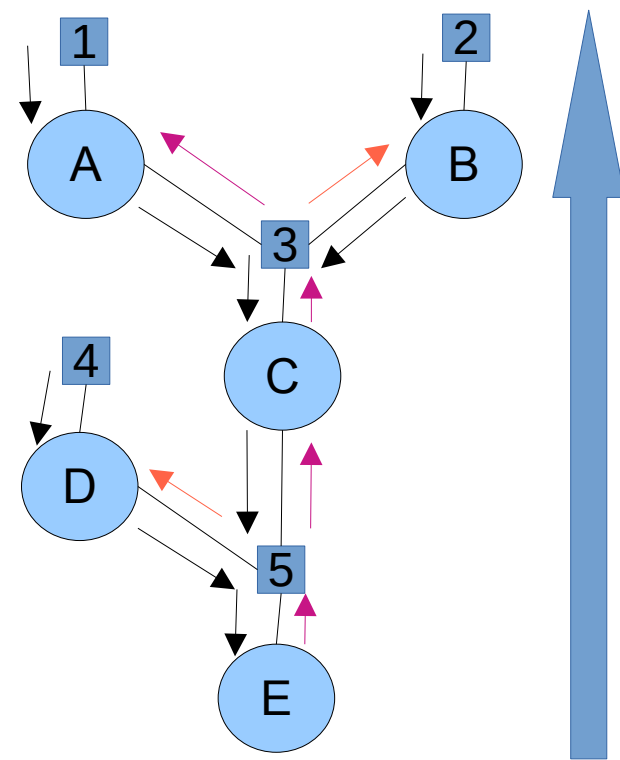


Message Passing on Factor Graphs

Calculating $P(A=a)$?

$$\rightarrow P(A = a) = \mu_{1 \rightarrow A}(a) \mu_{3 \rightarrow A}(a) / \left(\sum_{a'} \mu_{1 \rightarrow A}(a') \mu_{3 \rightarrow A}(a') \right)$$

Calculating all marginals:
messages to all other variables



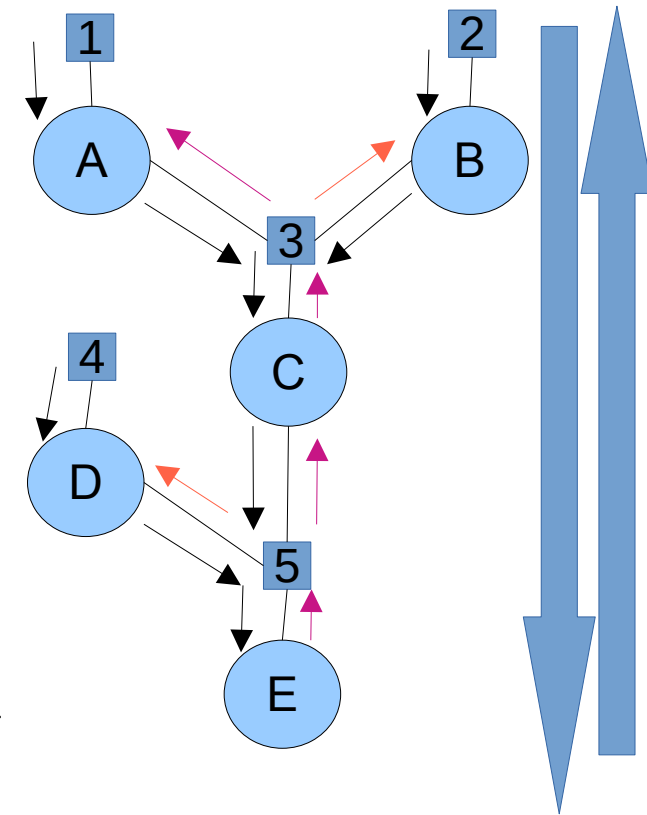
Belief Propagation on Factor Graphs that are trees



Judea Pearl, 1982; Turing Award winner

- 1) Pick some root
- 2) First iteration: send messages towards this root
- 3) Second iteration: send messages away from this root
- 4) Calculate the marginal of each node from the resulting messages

Only two passes needed, in polynomial time!



Message Passing on Factor Graphs

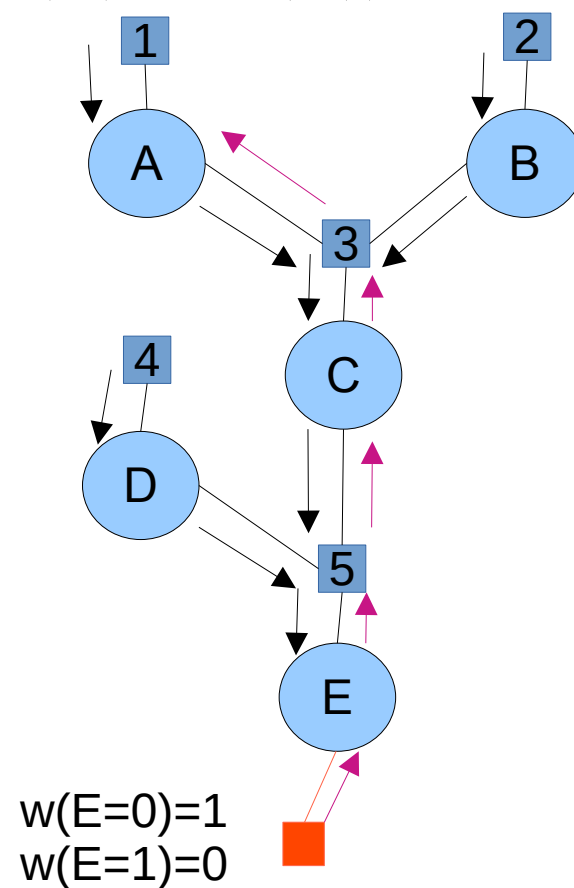
Calculating $P(A=a|E=0)$?

$$\rightarrow P(A = a|E = 0) = \mu_{1 \rightarrow A}(a) \mu_{3 \rightarrow A}(a) / \left(\sum_{a'} \mu_{1 \rightarrow A}(a') \mu_{3 \rightarrow A}(a') \right)$$

with modified factor graph to the right, in which additional factors are added to encode evidence.

These factors remove from the sum all cases in which $E=0$.

Note that normalisation of the resulting sum is really necessary!

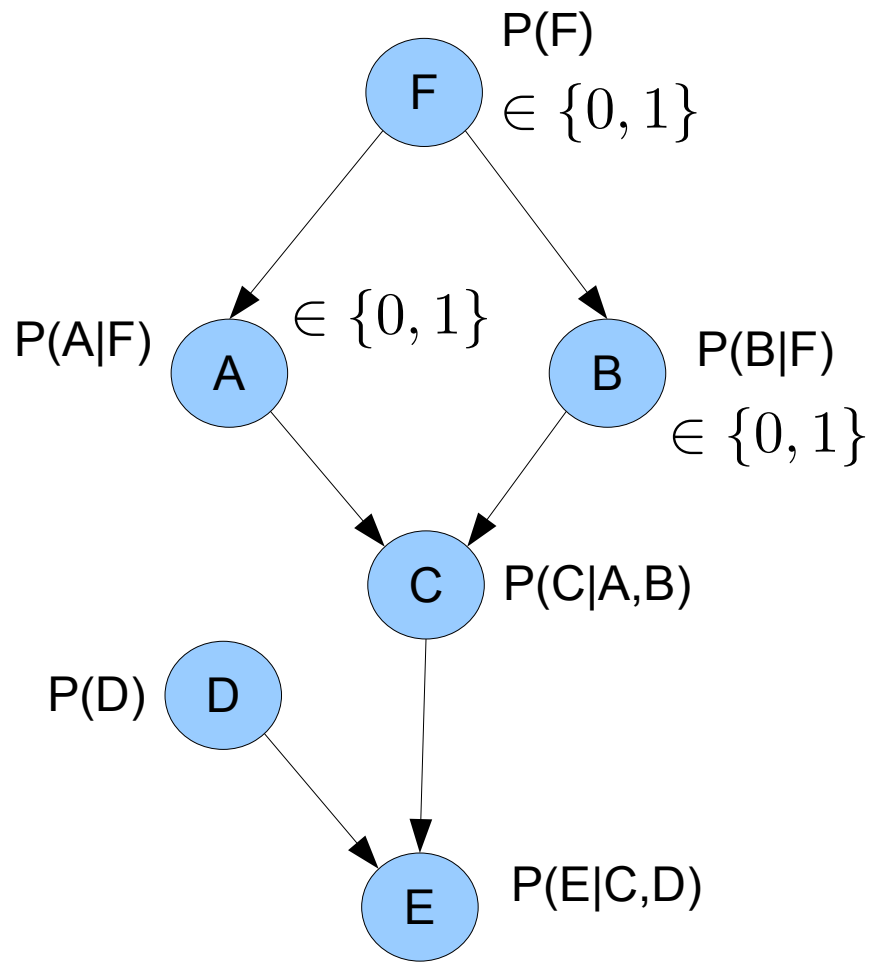


Loopy Belief Propagation

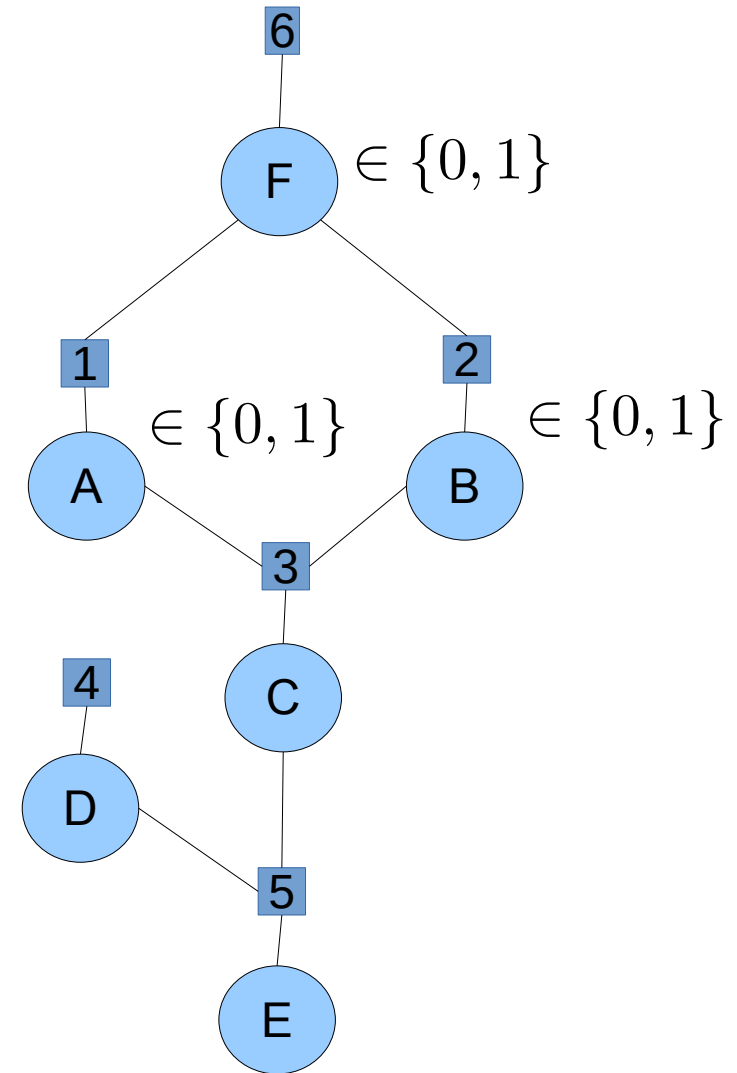
- What if the graph is not a tree?
- One algorithm:
 - 1) Initialize all messages to 1
 - 2) Then apply the equations of belief propagation, each time picking one of the edges, and updating the messages
 - 3) After a large number of iterations, we can use these messages to calculate the probabilities
- This algorithm has no guarantees whatsoever, but has been found to work well in practice

Exact Inference on Factor Graphs that are Not Trees

First: an example



Bayesian Network



Factor Graph

Exact Inference on Factor Graphs that are Not Trees

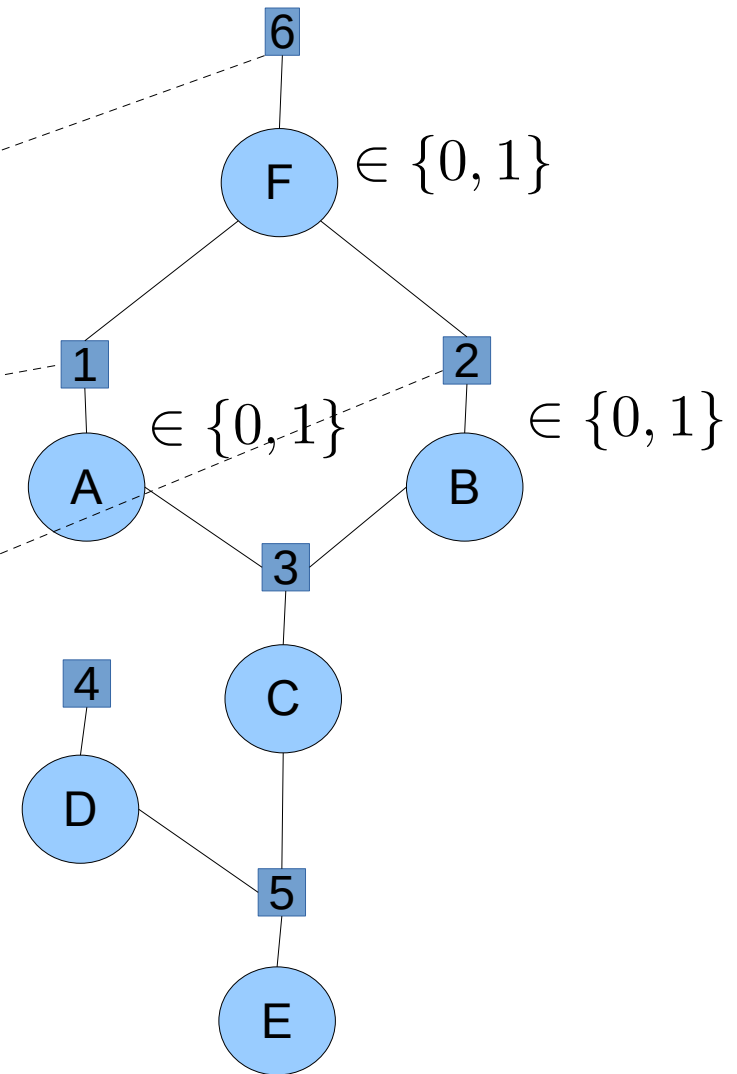
First: an example

With these tables providing the weights for factors

F	0	1
Weight	0.2	0.8

F	0	0	1	1
A	0	1	0	1
Weight	0.3	0.7	0.1	0.9

F	0	0	1	1
B	0	1	0	1
Weight	0.4	0.6	0.5	0.5



Factor Graph

Exact Inference on Factor Graphs that are Not Trees

First: an example

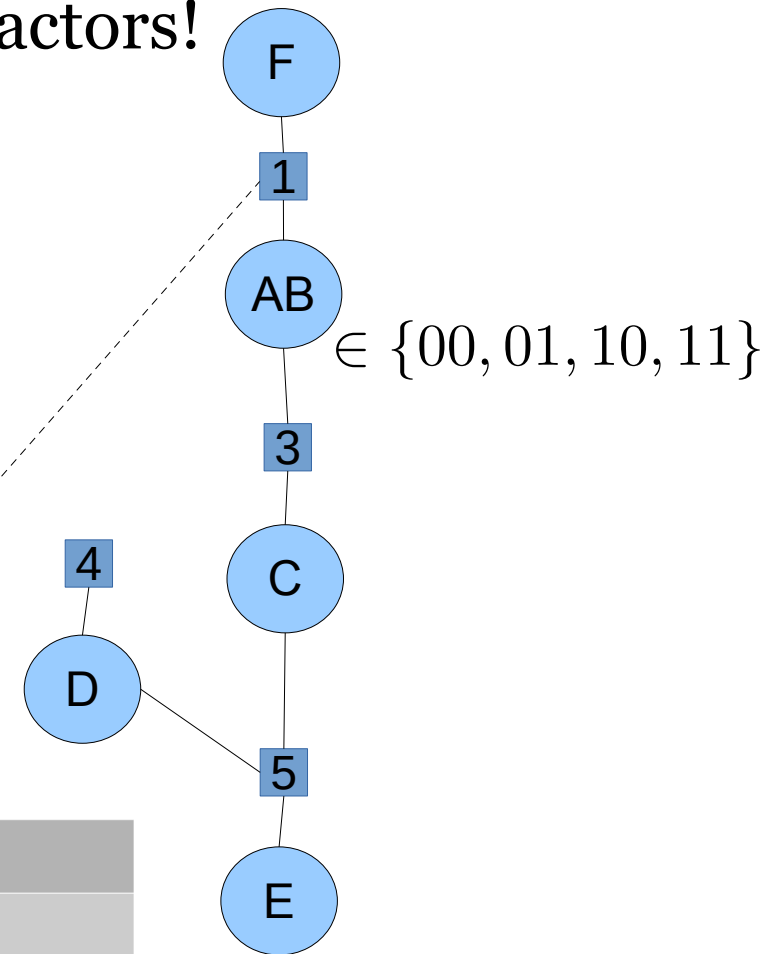
Let's simplify the graph and change the factors!

F	0	1
Weight	0.2	0.8

F	0	0	1	1
A	0	1	0	1
Weight	0.3	0.7	0.1	0.9

F	0	0	1	1
B	0	1	0	1
Weight	0.4	0.6	0.5	0.5

F	0	0	0	0	1	1	1	1
AB	00	01	10	11	00	01	10	11
Weight	0.2*	0.2*	0.2*	0.2*	0.8*	0.8*	0.8*	0.8*
	0.3*	0.3*	0.7*	0.7*	0.1*	0.1*	0.9*	0.9*
	0.4	0.6	0.4	0.6	0.5	0.5	0.5	0.5



Factor Graph

Exact Inference on Factor Graphs that are Not Trees

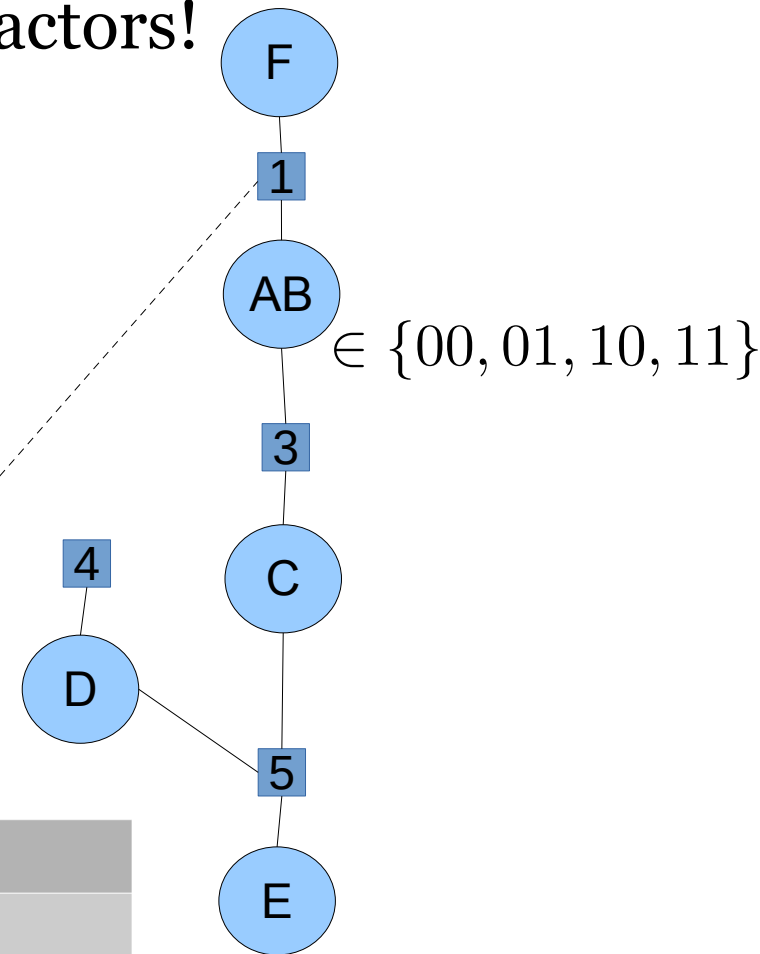
First: an example

Let's simplify the graph and change the factors!

Now we can calculate for instance

$$P(A=0) = P(AB=00) + P(AB=01)$$

→ we can calculate the marginal probability exactly using BP on this tree

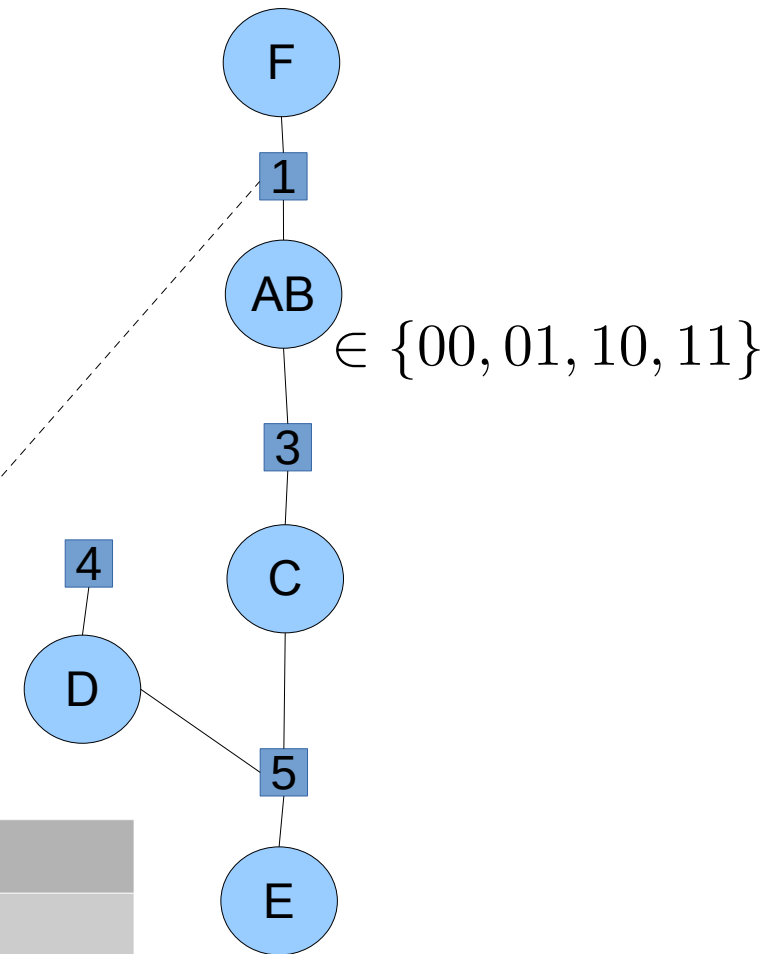


F	0	0	0	0	1	1	1	1
AB	00	01	10	11	00	01	10	11
Weight	0.2*	0.2*	0.2*	0.2*	0.8*	0.8*	0.8*	0.8*
	0.3*	0.3*	0.7*	0.7*	0.1*	0.1*	0.9*	0.9*
	0.4	0.6	0.4	0.6	0.5	0.5	0.5	0.5

Exact Inference on Factor Graphs that are Not Trees

Observations:

- Some graphs are “almost trees” and can be changed into trees by creating “larger” factors
- The weight tables associated to factors become larger; the larger the tables become, the more time consuming inference becomes
- How to determine whether a graph is almost a tree?

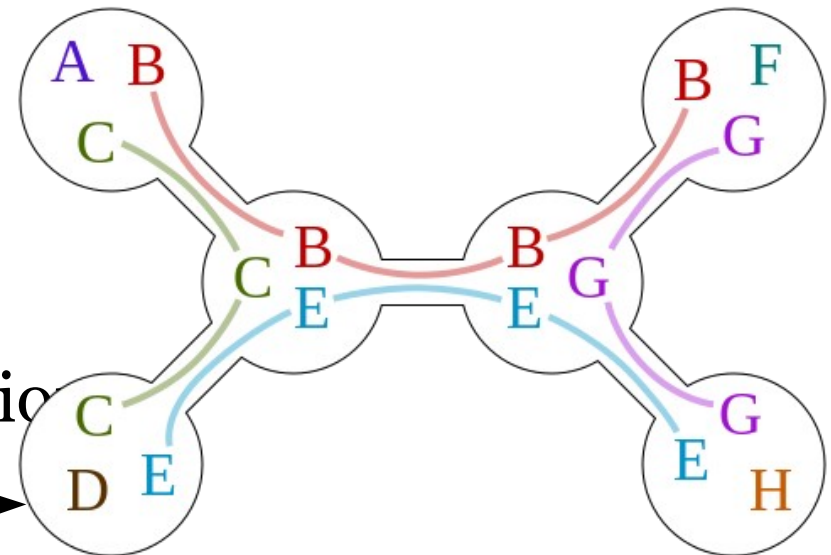
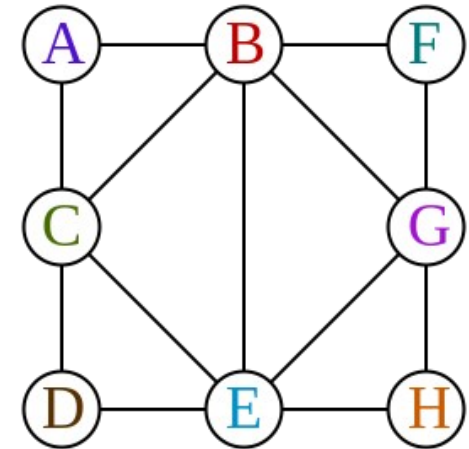


F	0	0	0	0	1	1	1	1
AB	00	01	10	11	00	01	10	11
Weight	0.2*	0.2*	0.2*	0.2*	0.8*	0.8*	0.8*	0.8*
	0.3*	0.3*	0.7*	0.7*	0.1*	0.1*	0.9*	0.9*
	0.4	0.6	0.4	0.6	0.5	0.5	0.5	0.5

Factor Graph

Bounded Tree Width

- The tree decomposition of a graph G is a tree T with nodes such that
 - ♦ every node in T corresponds to a subset of nodes in the graph G
 - ♦ for every edge in G , there is at least one node in T that contains both end points
 - ♦ if an original node is included in multiple nodes in T , this subset of nodes of T constitutes a connected tree
- The width of a tree decomposition is the maximum number of nodes in a node of the decomposition

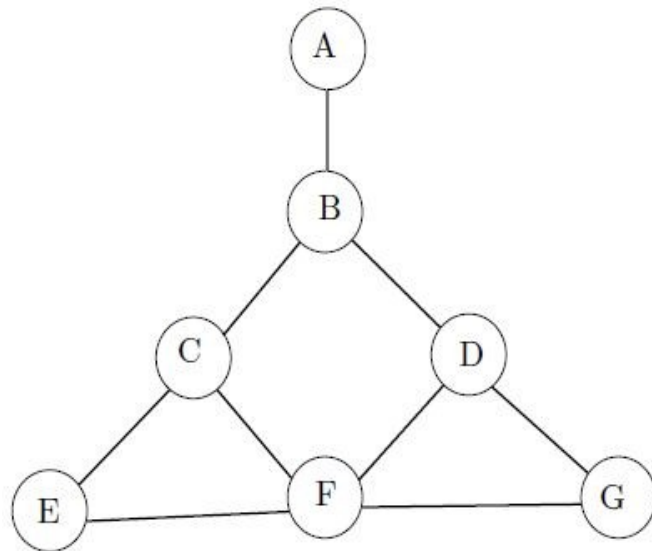


Tree width = 2 →

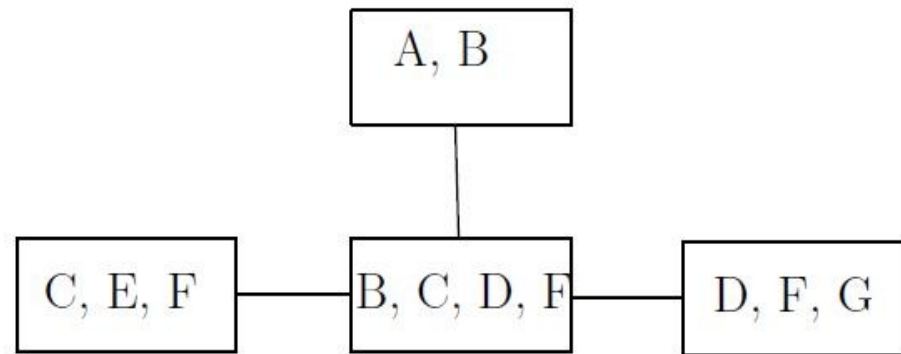
− 1

Bounded Tree Width

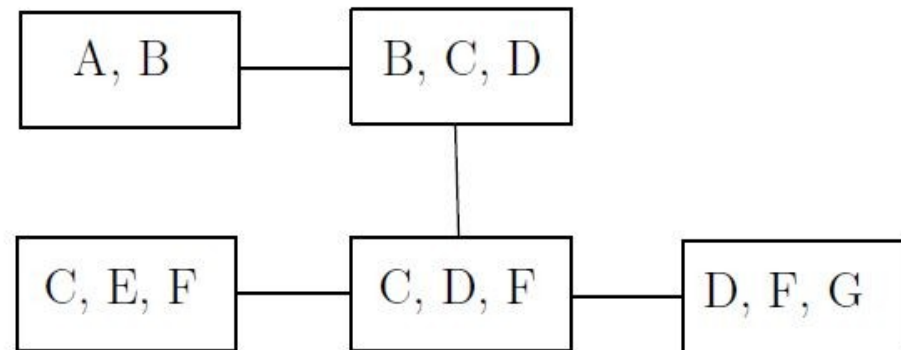
- The tree width of a graph is determined by the tree decomposition with the smallest width



The original graph G



A tree-decomposition of width 3



A tree-decomposition of width 2

Calculating Tree Decompositions

- Finding the tree width of a graph is an NP hard problem
- But huge numbers of approximation algorithms exist

Approximation	$f(k)$	$g(n)$	reference
exact	1	$O(n^{k+2})$	Arnborg, Corneil & Proskurowski (1987)
$4k + 3$	$O(3^{3k})$	$O(n^2)$	Robertson & Seymour (1995)
$8k + 7$	$2^{O(k \log n)}$	$O(n \log^2 n)$	Lagergren (1996)
$5k + 4$ (or $7k + 6$)	$2^{O(k \log n)}$	$O(n \log n)$	Reed (1992)
exact	$2^{O(k^3)}$	$O(n)$	Bodlaender (1996)
$O(k \cdot \sqrt{\log k})$	1	$n^{O(1)}$	Feige, Hajiaghayi & Lee (2008)
$4.5k + 4$	2^{3k}	$O(n^2)$	Amir (2010)
$\frac{11}{3}k + 4$	$2^{3.6982k}$	$O(n^3 \log^4 n)$	Amir (2010)
exact	1	$O(1.7347^n)$	Fomin, Todinca & Villanger (2015)
$3k + 2$	$2^{O(k)}$	$O(n \log n)$	Bodlaender et al. (2016)
$5k + 4$	$2^{O(k)}$	$O(n)$	Bodlaender et al. (2016)
k^2	k^7	$O(n \log n)$	Fomin et al. (2018)
$5k + 4$	$2^{8.765k}$	$O(n \log n)$	Belbasi & Fürer (2021a)
$2k + 1$	$2^{O(k)}$	$O(n)$	Korhonen (2021)
$5k + 4$	$2^{6.755k}$	$O(n \log n)$	Belbasi & Fürer (2021b)
exact	$2^{O(k^2)}$	$O(n^4)$	Korhonen & Lokshtanov (2023)
$(1 + \varepsilon)k$	$k^{O(k/\varepsilon)}$	$O(n^4)$	Korhonen & Lokshtanov (2023)

From Wikipedia

e.g. Robertson & Seymour (1995)

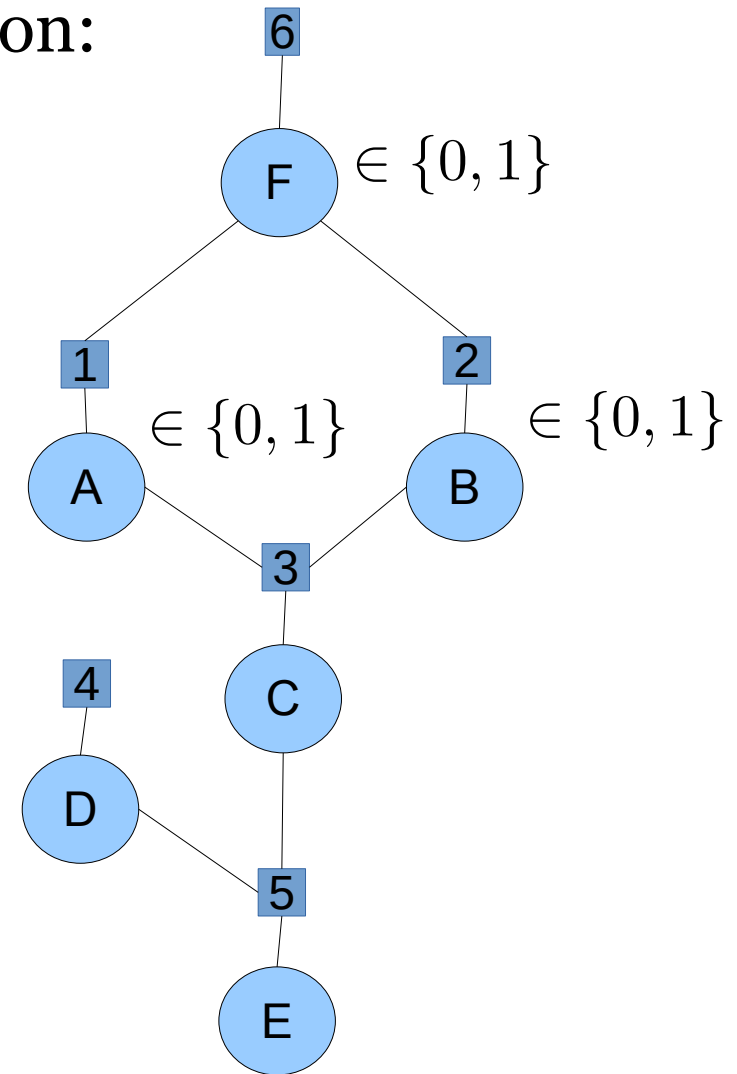
For a treewidth of k and a graph of n nodes, the algorithm will within time $O(f(k) \cdot g(n)) = O(3^{3k} n^2)$ either

- say that no decomposition of width k exist
- or return one of at most $4k+3$

Tree Width for Factor Graphs

Steps for determining a tree decomposition:

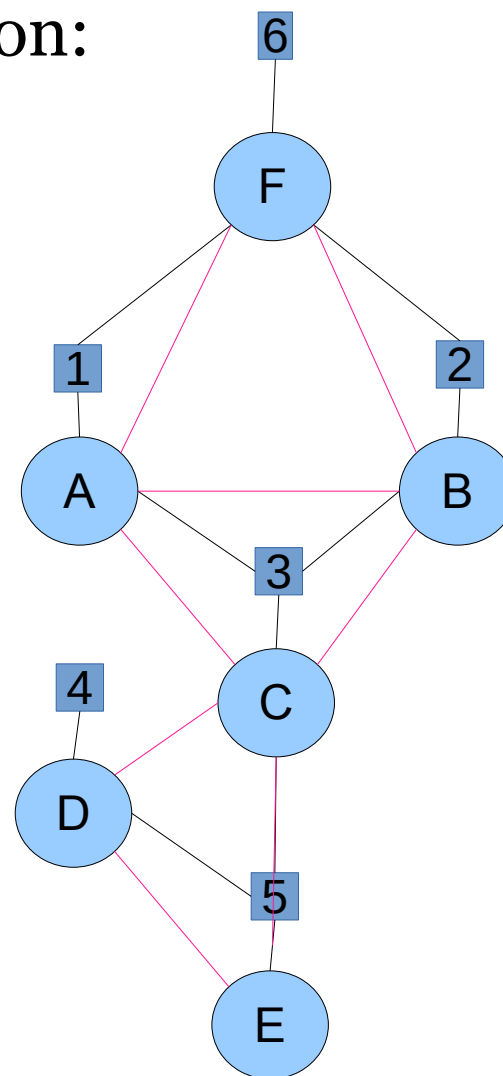
- 1) Replace factors by *cliques*, by adding edges between all pairs of variables connected by a factor



Tree Width for Factor Graphs

Steps for determining a tree decomposition:

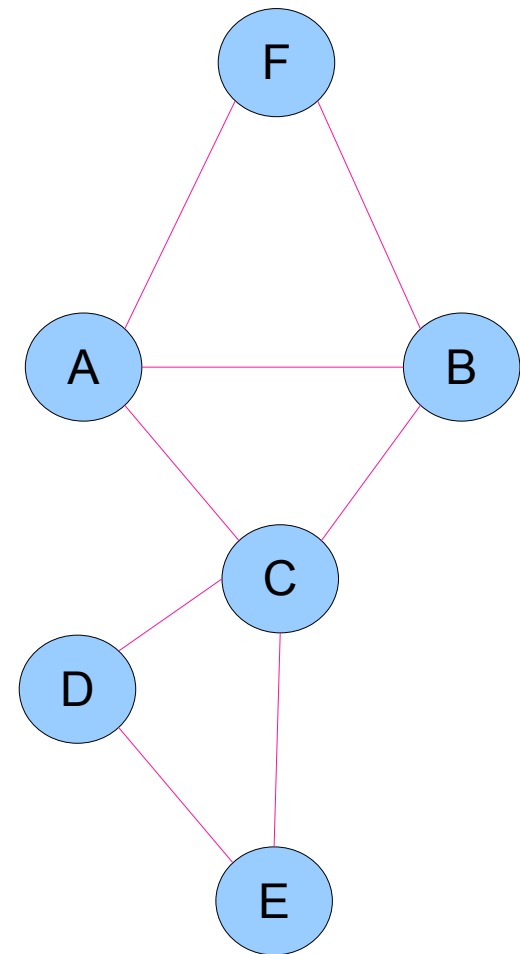
- 1) Replace factors by *cliques*, by adding edges between all pairs of variables connected by a factor



Tree Width for Factor Graphs

Steps for determining a tree decomposition:

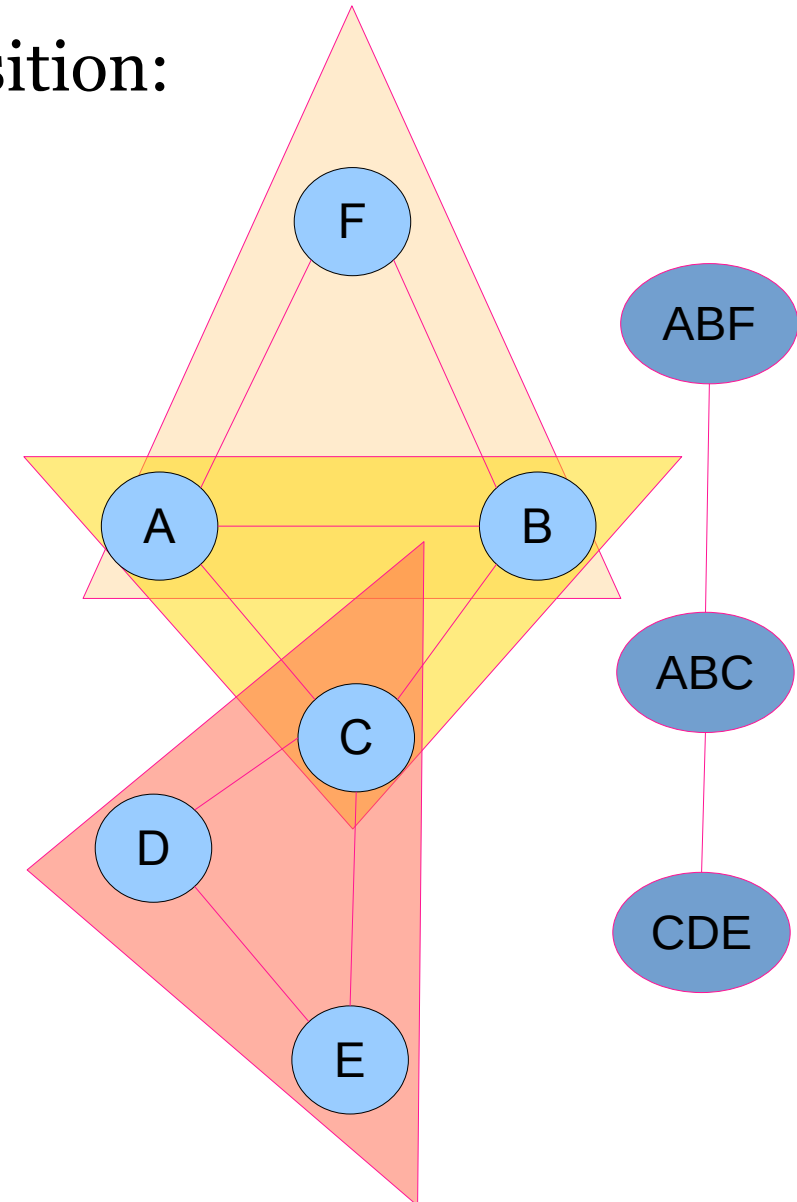
- 1) Replace factors by *cliques*, by adding edges between all pairs of variables connected by a factor (*moral graph*)



Tree Width for Factor Graphs

Steps for determining a tree decomposition:

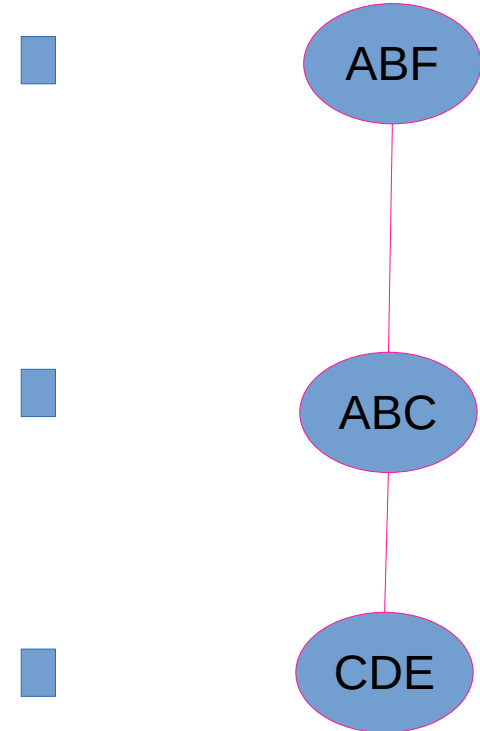
- 1) Replace factors by *cliques*, by adding edges between all pairs of variables connected by a factor (*moral graph*)
- 2) Calculate a tree decomposition (of low width)



Exact Inference using Tree Decomposition

From the tree decomposition we can create a new factor graph. Follow these steps:

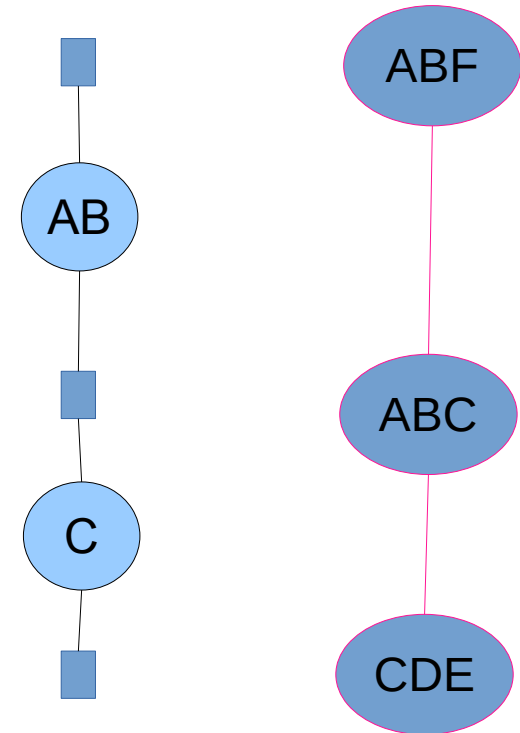
- 1) Turn nodes of the decomposition into factors



Exact Inference using Tree Decomposition

From the tree decomposition we can create a new factor graph. Follow these steps:

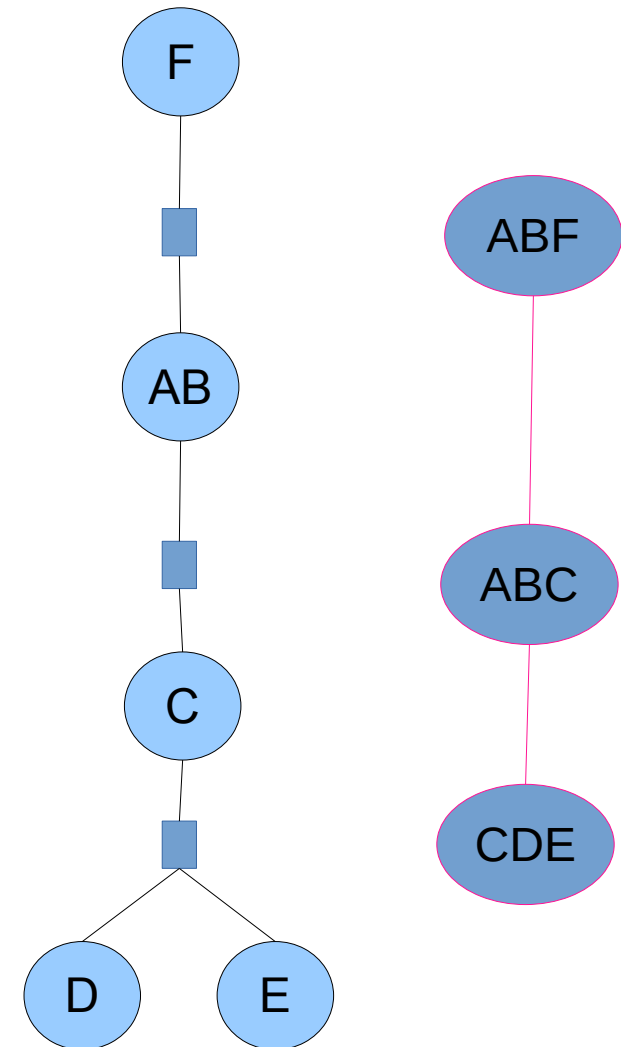
- 1) Turn nodes of the decomposition into factors
- 2) Turn edges of the decomposition into new variables; the new variable represents the *sepset*: original variables common to two factors



Exact Inference using Tree Decomposition

From the tree decomposition we can create a new factor graph. Follow these steps:

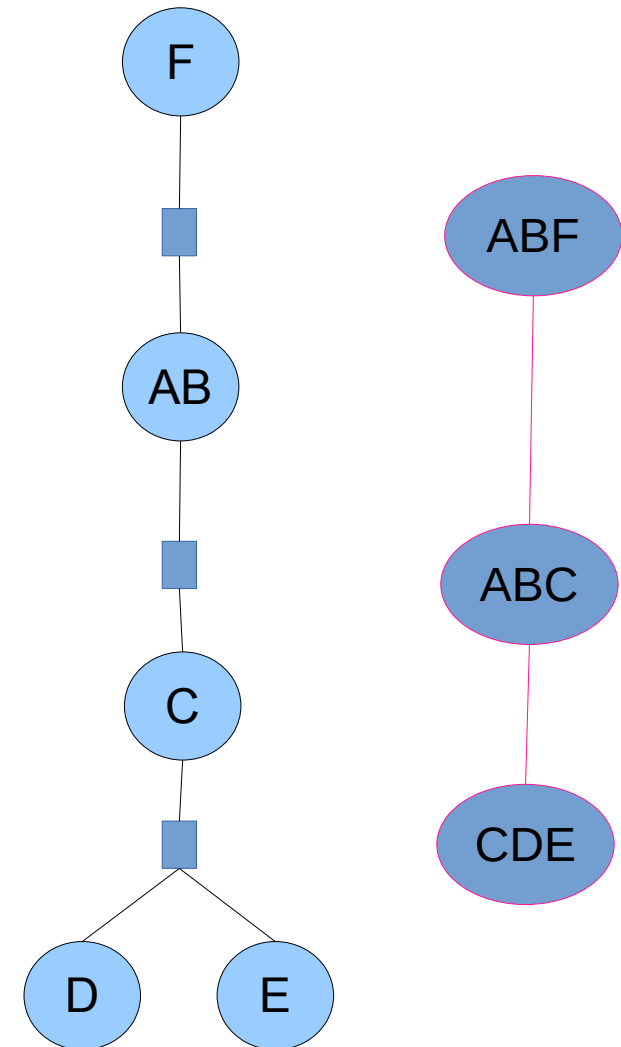
- 1) Turn nodes of the decomposition into factors
- 2) Turn edges of the decomposition into new variables; the new variable represents the *sepset*: original variables common to two factors
- 3) Add new variables for original variables present in only 1 factor



Exact Inference using Tree Decomposition

From the tree decomposition we can create a new factor graph. Follow these steps:

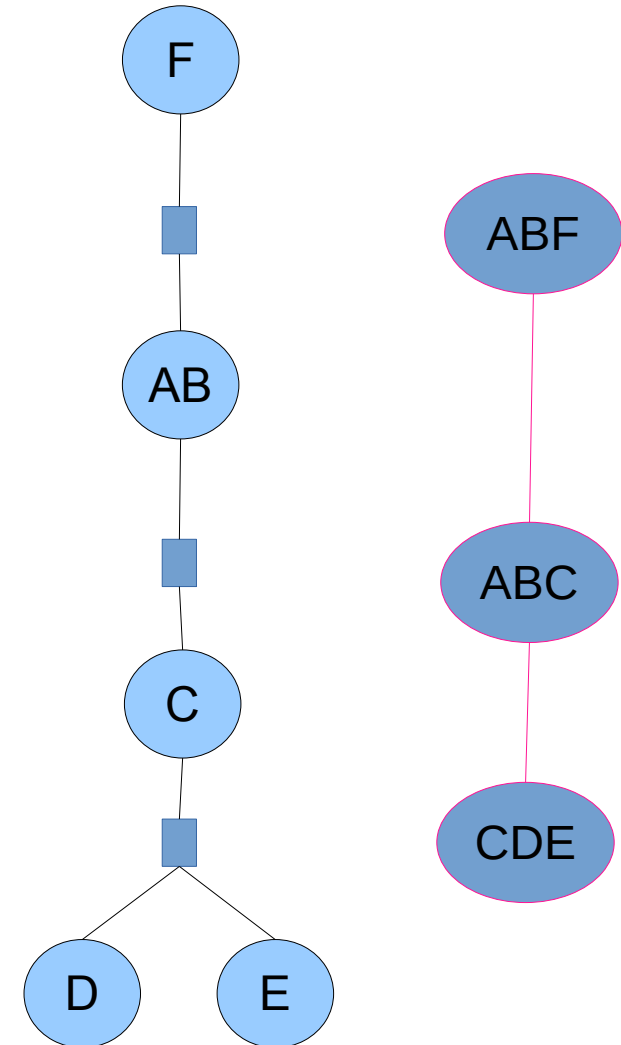
- 1) Turn nodes of the decomposition into factors
- 2) Turn edges of the decomposition into new variables; the new variable represents the *sepset*: original variables common to two factors
- 3) Add new variables for original variables present in only 1 factor
- 4) Define weights for new factors



Exact Inference using Tree Decomposition

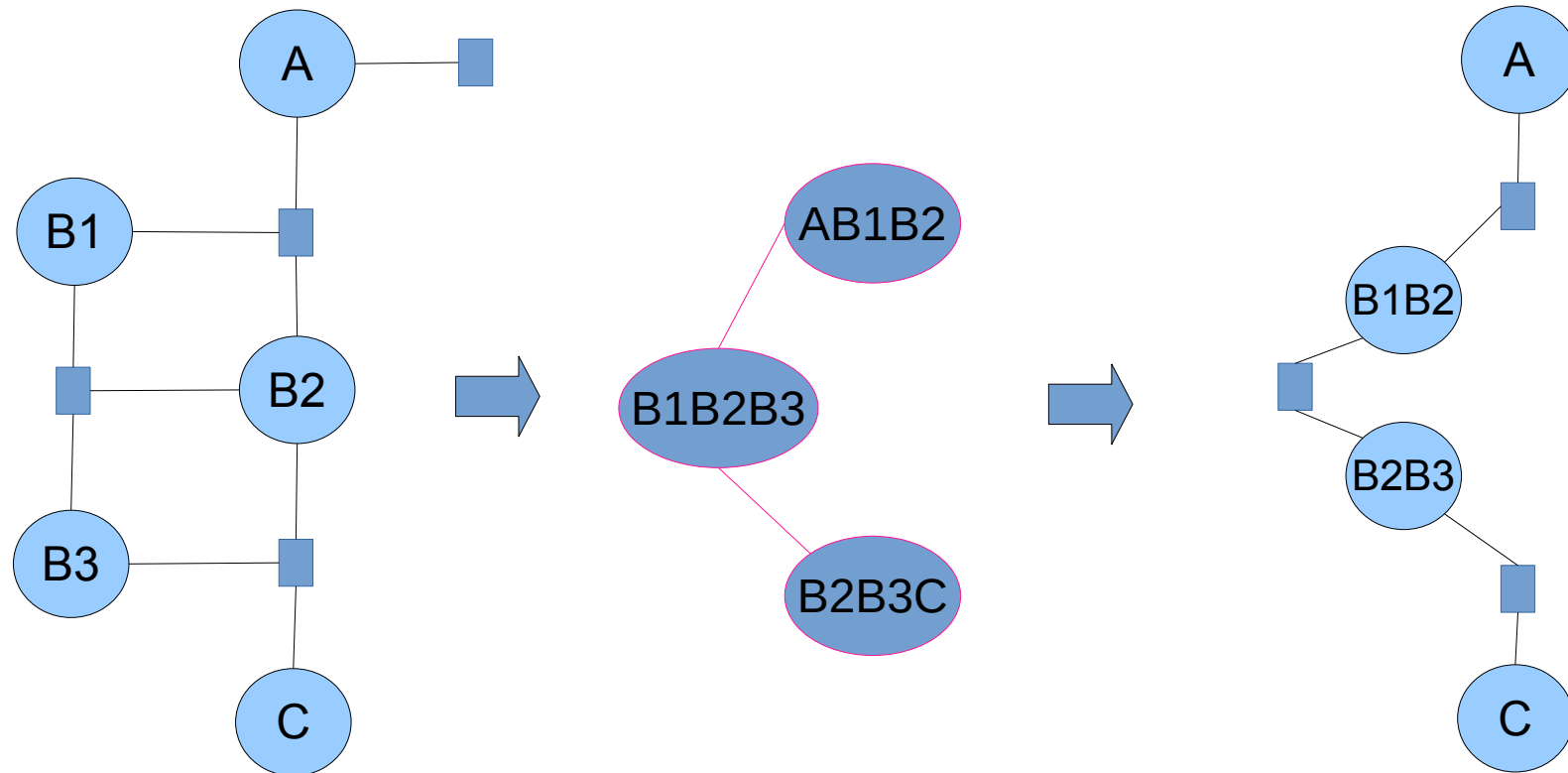
First some observations:

- By definition the resulting factor graph is a tree
- Every original factor is fully included in at least one new factor
- The maximal number of original variables included in one factor corresponds to *tree width* + 1
- One original variable can be included in *multiple* new variables of the factor graph



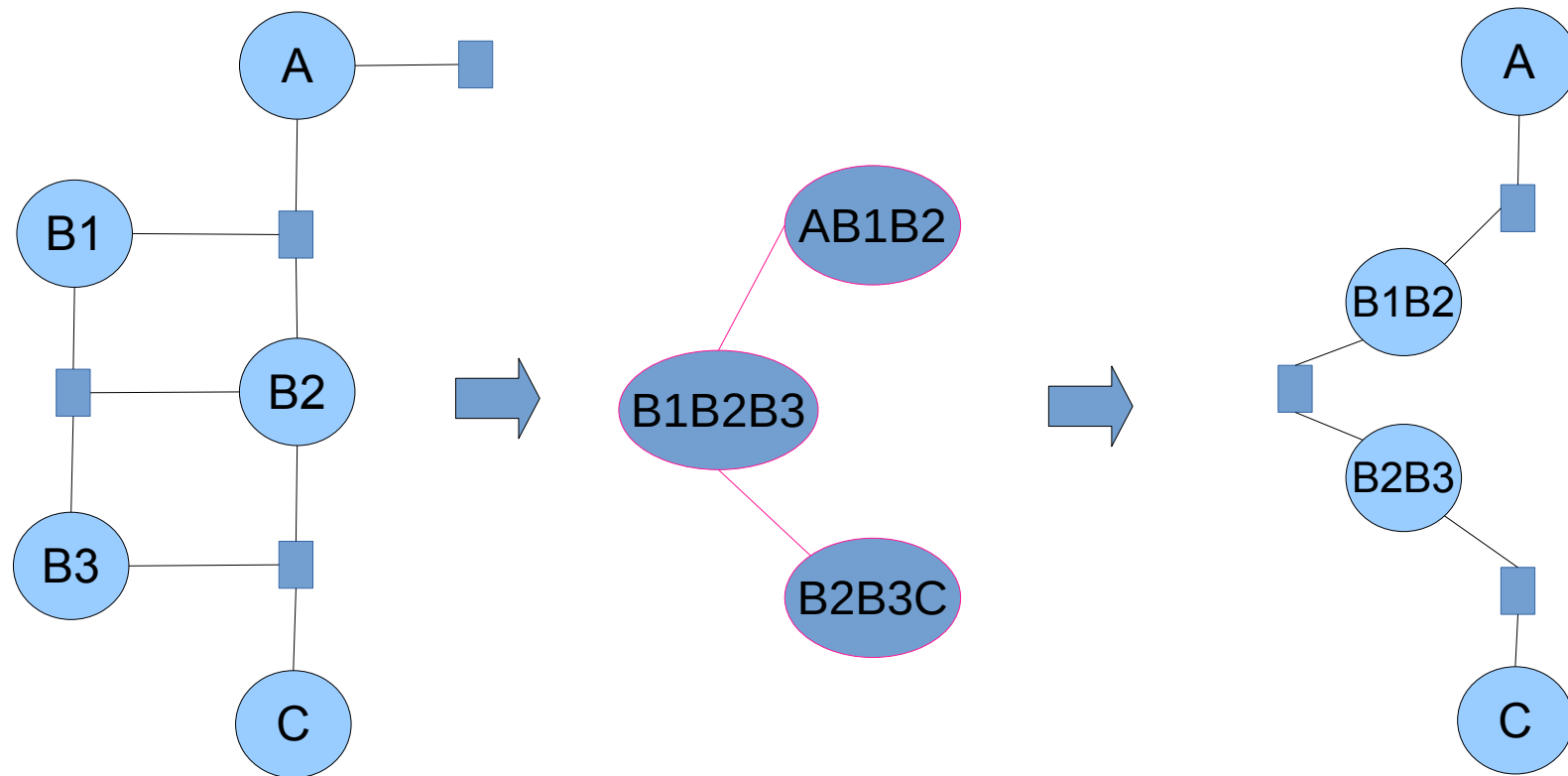
Exact Inference using Tree Decomposition

One original variable can be included in *multiple* new variables of the factor graph



Exact Inference using Tree Decomposition

One original variable can be included in *multiple* new variables of the factor graph

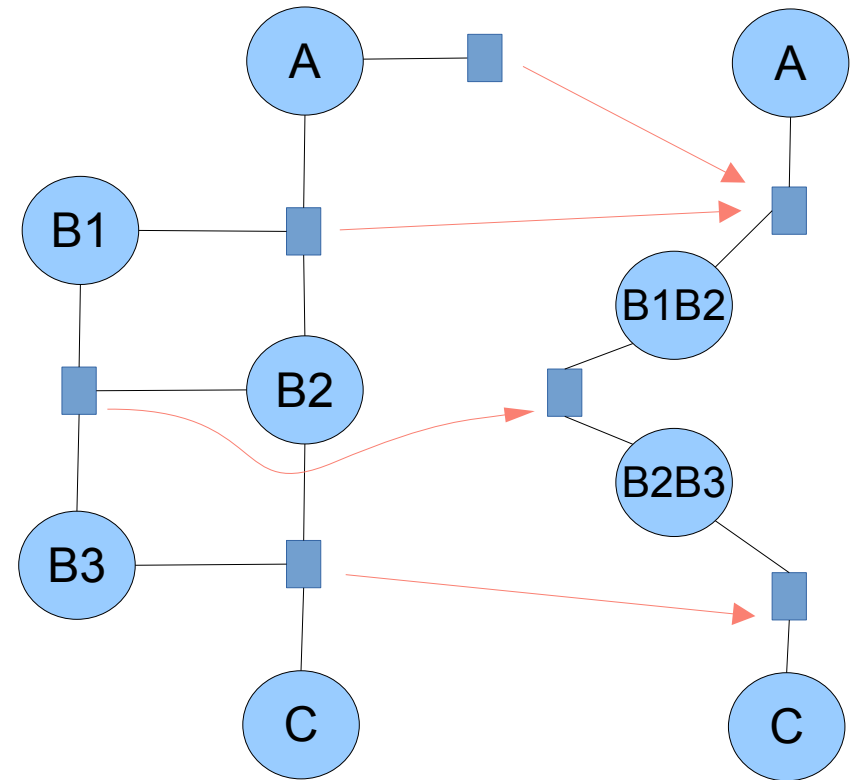


However: then there is a new factor that connects these new variables

Exact Inference using Tree Decomposition

Definition of weights:

- 1) For every original factor, pick one new factor in which we will include this factor
- 2) If a factor connects two new variables having a same original variable in common, this new factor will give weight 0 to assignments that are incompatible



$$w_{new}(B1B2 = b1b2, A = a) = w_{orig}(B1 = b1, B2 = b2, A = a) \times w_{orig}(A = a)$$

$$w_{new}(B1B2 = b1b2, B2B3 = b2'b3) = \begin{cases} w_{orig}(B1 = b1, B2 = b2, B3 = b3) & \text{if } b2 = b2' \\ 0 & \text{otherwise} \end{cases}$$

Exact Inference using Tree Decomposition

The new factor graph describes the same distribution:

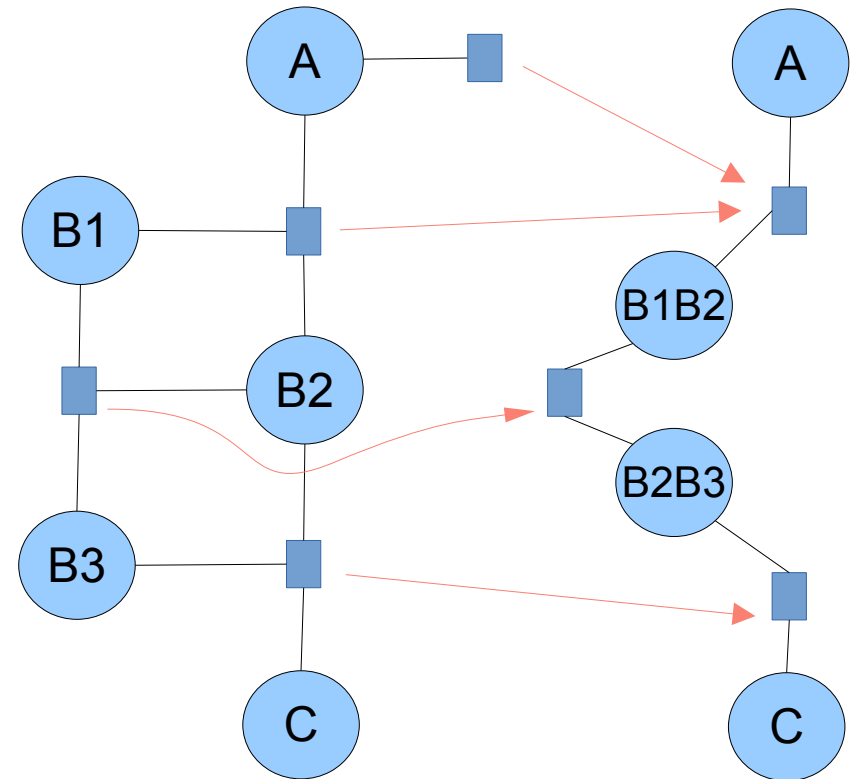
for any assignment it holds that

$$w_{old}(A, B_1, B_2, B_3, C) = w_{new}(A, B_1B_2, B_2B_3, C)$$

and

$$w_{new}(A, B_1B_2, B_2B_3, C) = 0$$

for any B_1B_2, B_2B_3 in which B_2 has a different assignment in B_1B_2 than in B_2B_3



Exact Inference using Tree Decomposition

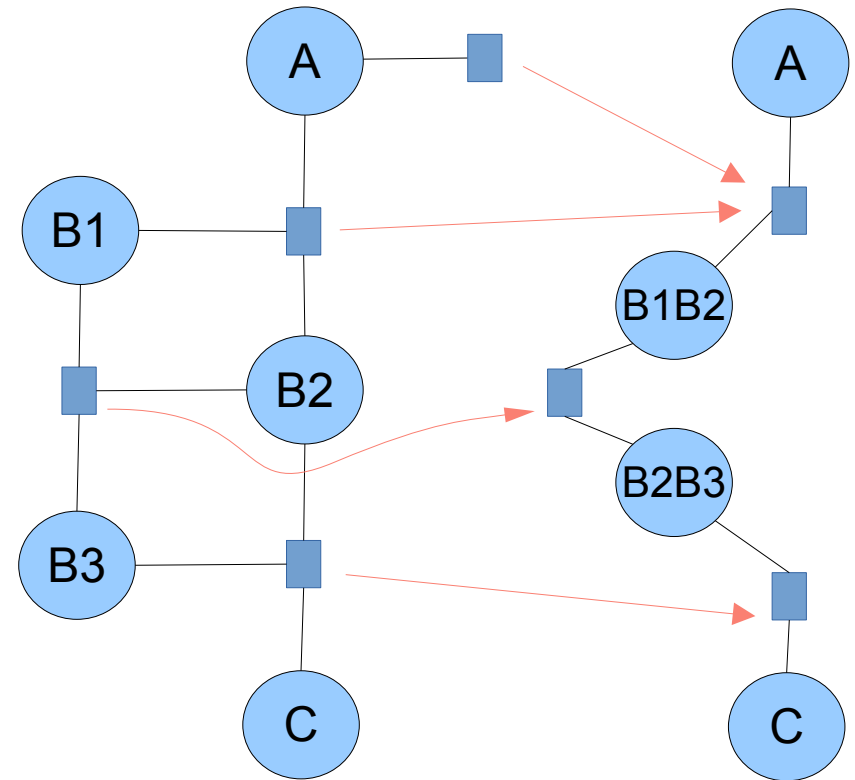
For instance, to calculate

$P(B_1=1)$

the equality

$$P(B_1=1) = P(B_1=1, B_2=1) + P(B_1=1, B_2=0)$$

can be used; using BP these probabilities are calculated



Exact Inference using Tree Decomposition

Summarizing:

- For a given Factor Graph: calculate its tree decomposition
- Using this tree decomposition, determine a new factor graph
- Perform belief propagation on this new factor graph

Advantages:

- Allows to perform exact inference efficiently

Disadvantages:

- Complex to understand & implement
- Only useful for factor graphs of low tree width

Exact Inference using Tree Decomposition

Other comments:

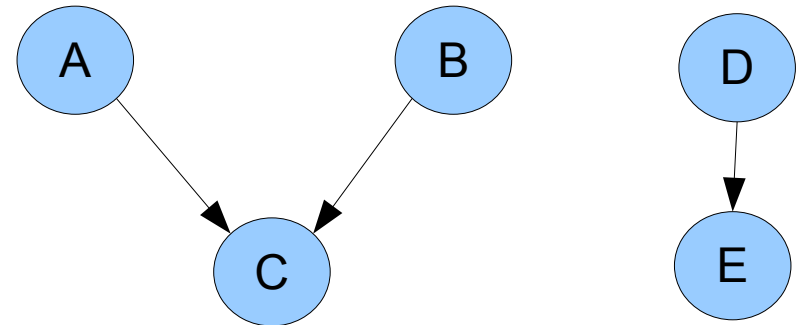
- A tree decomposition is also referred to as a *clique tree* or a *junction tree*
- Belief propagation on a tree decomposition is also referred to as the *junction tree algorithm* – there are a number of different ways of formalizing this algorithm
- All these algorithms can be adapted to other forms of inference; for instance, to calculate the probability of the most probable assignment to all variables, the *sum* operator is replaced by the *max* operator
- Can also be applied on *constraint programming*
 - ◆ However, in CP the treewidth is often very high
 - ◆ Reflect on how to do this for CNF formulas (SAT)

Learning Graphical Models

Bayesian Networks: Learning Parameters

Given a fixed *structure*, learning *maximum likelihood* parameters is relatively easy

A	B	C	D	E
0	1	0	0	0
0	0	0	0	1
1	0	1	0	0
1	0	0	0	1
0	1	1	0	0
0	0	0	1	1
0	0	1	1	1
1	1	1	0	0
1	1	0	0	1
1	1	1	0	1



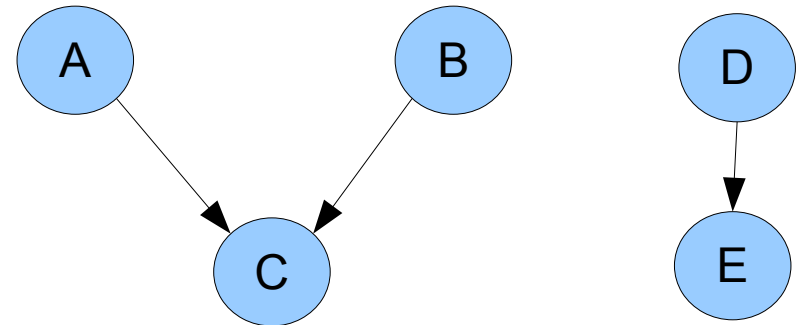
Find parameters such that

$P(A=0, B=1, C=0, D=0, E=0) *$
 $P(A=0, B=0, C=0, D=0, E=1) *$
 $P(A=1, B=0, C=1, D=0, E=0) *$
 $P(A=1, B=0, C=0, D=0, E=1) *$
 $P(A=0, B=1, C=1, D=0, E=0) *$
 $P(A=0, B=0, C=0, D=1, E=1) *$
 $P(A=0, B=0, C=1, D=1, E=1) *$
 $P(A=1, B=1, C=1, D=0, E=0) *$
 $P(A=1, B=1, C=0, D=0, E=1) *$
 $P(A=1, B=1, C=1, D=0, E=1)$ is maximal

Bayesian Networks: Learning Parameters

Given a fixed *structure*, learning *maximum **log**-likelihood* parameters is relatively easy

A	B	C	D	E
0	1	0	0	0
0	0	0	0	1
1	0	1	0	0
1	0	0	0	1
0	1	1	0	0
0	0	0	1	1
0	0	1	1	1
1	1	1	0	0
1	1	0	0	1
1	1	1	0	1



Find parameters such that

$\log(P(A=0, B=1, C=0, D=0, E=0)) +$
 $\log(P(A=0, B=0, C=0, D=0, E=1)) +$
 $\log(P(A=1, B=0, C=1, D=0, E=0)) +$
 $\log(P(A=1, B=0, C=0, D=0, E=1)) +$
 $\log(P(A=0, B=1, C=1, D=0, E=0)) +$
 $\log(P(A=0, B=0, C=0, D=1, E=1)) +$
 $\log(P(A=0, B=0, C=1, D=1, E=1)) +$
 $\log(P(A=1, B=1, C=1, D=0, E=0)) +$
 $\log(P(A=1, B=1, C=0, D=0, E=1)) +$
 $\log(P(A=1, B=1, C=1, D=0, E=1))$ is maximal

Bayesian Networks: Learning Parameters

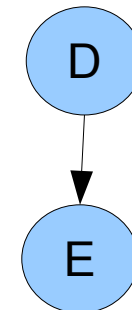
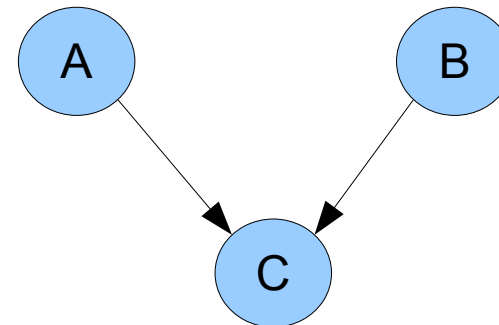
Example

A	B	C	D	E
0	1	0	0	0
0	0	0	0	1
1	0	1	0	0
1	0	0	0	1
0	1	1	0	0
0	0	0	1	1
0	0	1	1	1
1	1	1	0	0
1	1	0	0	1
1	1	1	0	1

0	1
0.5	0.5

0	1
0.5	0.5

0	1
0.8	0.2



A	B	C=0	C=1
0	0	2/3	1/3
0	1	1/2	1/2
1	0	1/2	1/2
1	1	1/3	2/3

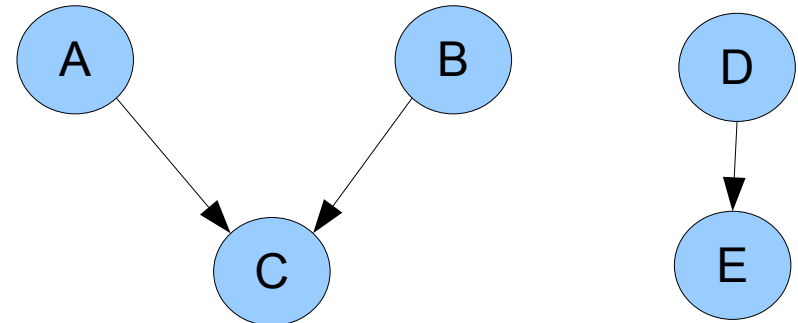
D	E=0	E=1
0	4/8	4/8
1	0/2	2/2

Bayesian Networks: Learning Parameters - Motivation

We wish to maximize

$$\begin{aligned}\log P(\mathcal{D}) &= \sum_{(a,b,c,d,e) \in \mathcal{D}} \log P(A = a, B = b, C = c, D = d, E = e) \\ &= \sum_{(a,b,c,d,e) \in \mathcal{D}} \log P(A = a)P(B = b)P(C = c|A = a, B = b)P(D = d)P(E = e|D = d) \\ &= \sum_{(a,b,c,d,e) \in \mathcal{D}} (\log P(A = a) + \log P(B = b) + \log P(C = c|A = a, B = b) + \\ &\quad \log P(D = d) + \log P(E = e|D = d)) \\ &= \sum_{(a,b,c,d,e) \in \mathcal{D}} \log P(A = a) + \cdots + \sum_{(a,b,c,d,e) \in \mathcal{D}} \log P(E = e|D = d)\end{aligned}$$

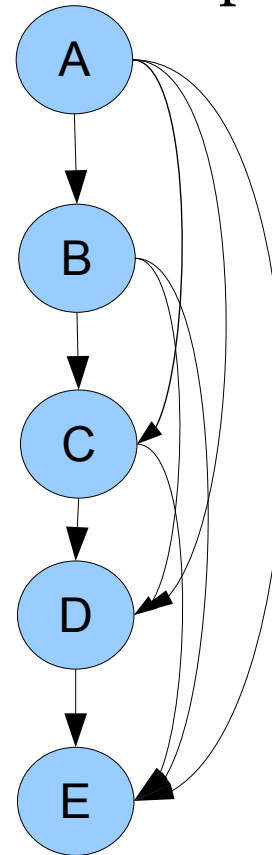
We can maximize each sum independently



Bayesian Networks: Learning Structure

- Learning structure can be hard
- General idea: find a structure that optimizes a score
- Using only (log-)likelihood could favour a complex type of network

A	B	C	D	E
0	1	0	0	0
0	0	0	0	1
1	0	1	0	0
1	0	0	0	1
0	1	1	0	0
0	0	0	1	1
0	0	1	1	1
1	1	1	0	0
1	1	0	0	1
1	1	1	0	1



Bayesian Networks: Learning Structure

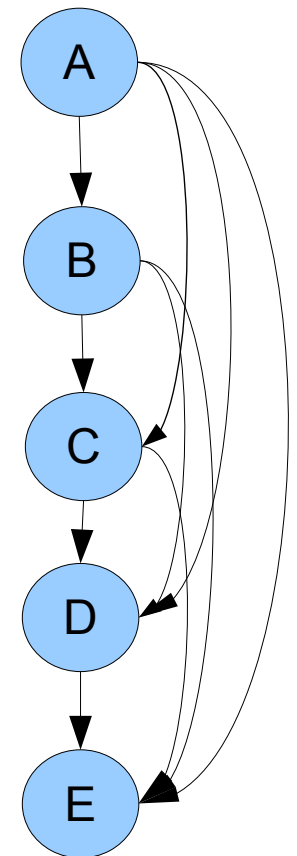
Solution: add penalty terms for complex networks, that is networks with many *parameters* (p)

0	1
0.8	0.2

1 parameter

$$\log P(\mathcal{D}) - \phi(|\mathcal{D}|) \cdot p$$

- *Akaike Information Criterion (AIC)*:
 $\varphi(n) = 1$
- *Bayesian Information Criterion (BIC)*:
 $\varphi(n) = \log(n)/2$

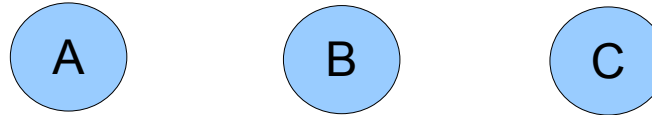


Algorithm 1: Greedy Search

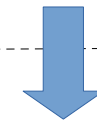
- Initialize a DAG G without edges and a node for every variable
- Let S = the score of G
- Repeat as long as S increases:
 - ◆ For every edge addition, deletion, or reversal in G , yielding a DAG:
 - Determine the new graph G' and its parameters
 - Calculate the new score S' of G'
 - If $S' > S$ then
 - $S = S'$
 - $G = G'$
- Return G

Algorithm 1: Greedy Search

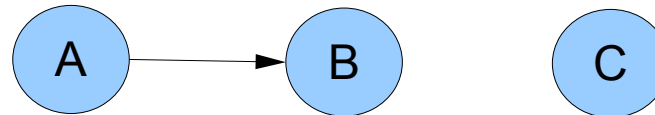
Iteration 1



Example addition



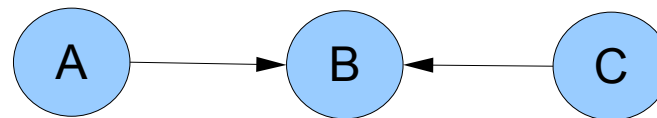
Iteration 2



Example addition



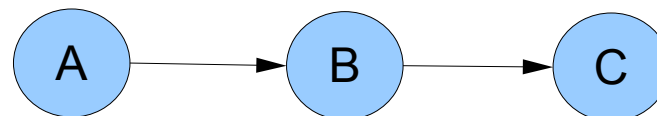
Iteration 3



Example reversal



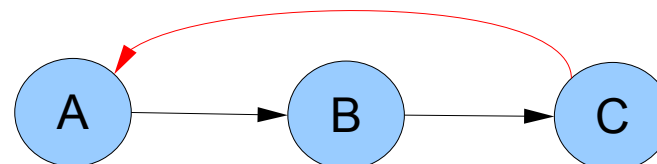
Iteration 4



Example **forbidden** addition



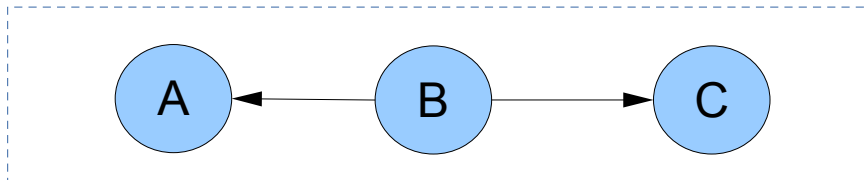
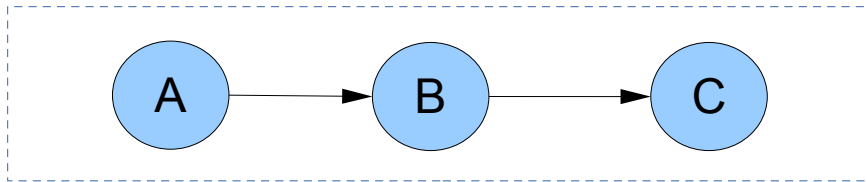
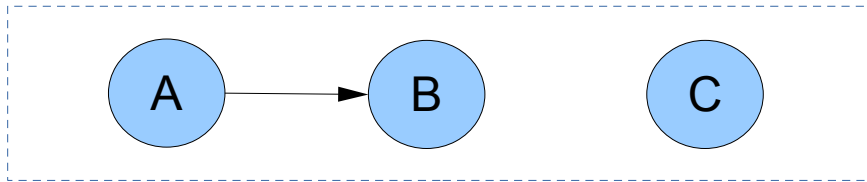
Iteration 5



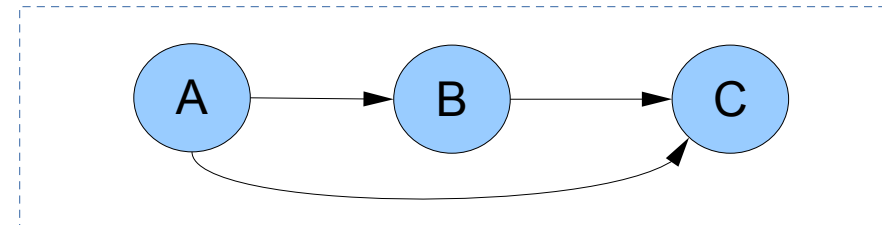
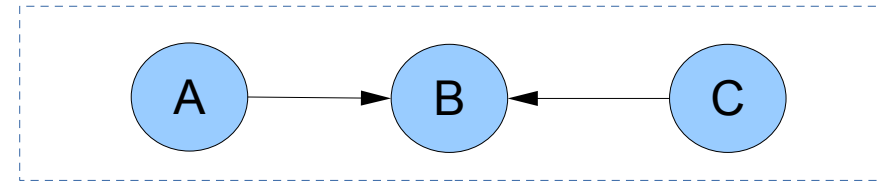
Algorithm 2: Chow-Liu Trees

Algorithm for finding the optimal BN in which every node has at most one parent

Can be found



Cannot be found

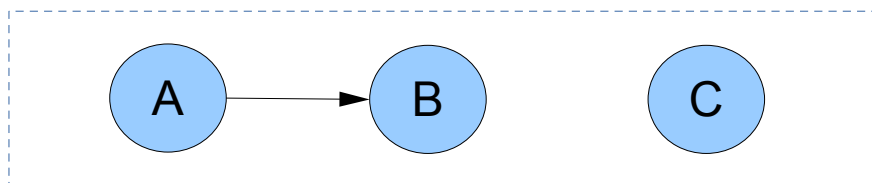


Algorithm 2: Chow-Liu Trees

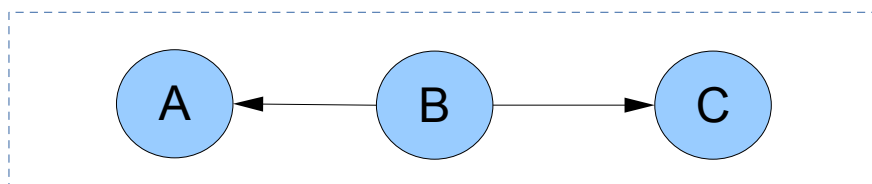
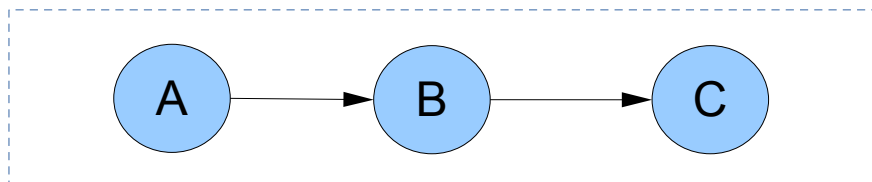
Intuitions

Short hand notation: $f(X = x) = \text{freq}(\{X = x\})$

$$\log P(A = a, B = b, C = c) =$$



$$\begin{aligned} & \log P(A = a) + \log P(C = c) + \log P(B = b|A = a) = \\ & \log f(A = a) + \log f(C = c) + \log f(A = a, B = b)/f(A = a) = \\ & \log f(A = a) + \log f(C = c) + \\ & \quad \log f(A = a, B = b) - \log f(A = a) + \log f(B = b) - \log f(B = b) = \\ & \log f(A = a) + \log f(B = b) + \log f(C = c) + \\ & \quad \log(f(A = a, B = b)/(f(A = a)f(B = b))) \end{aligned}$$

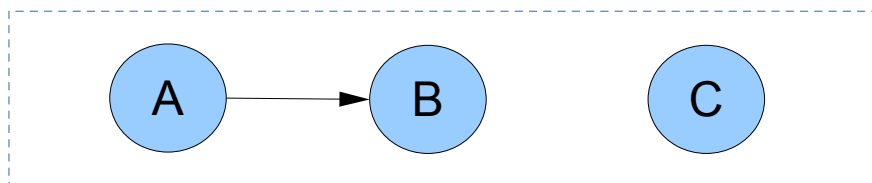


Algorithm 2: Chow-Liu Trees

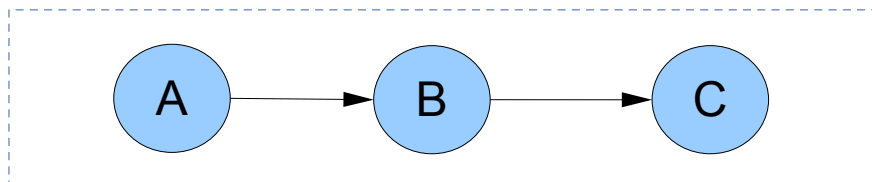
Intuitions

Short hand notation: $f(X = x) = \text{freq}(\{X = x\})$

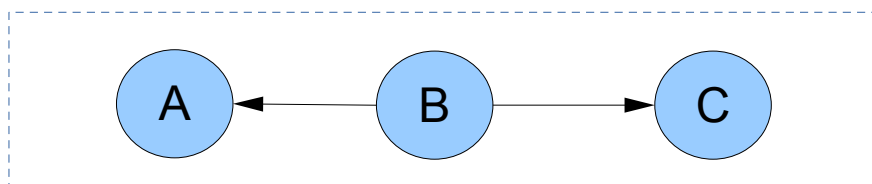
$$\log P(A = a, B = b, C = c) =$$



$$\begin{aligned} & \log P(A = a) + \log P(C = c) + \log P(B = b|A = a) = \\ & \log f(A = a) + \log f(C = c) + \log f(A = a, B = b)/f(A = a) = \\ & \log f(A = a) + \log f(C = c) + \\ & \quad \log f(A = a, B = b) - \log f(A = a) + \log f(B = b) - \log f(B = b) = \\ & \log f(A = a) + \log f(B = b) + \log f(C = c) + \\ & \quad \log(f(A = a, B = b)/(f(A = a)f(B = b))) \end{aligned}$$



$$\begin{aligned} & \log f(A = a) + \log f(B = b) + \log f(C = c) + \\ & \quad \log(f(A = a, B = b)/(f(A = a)f(B = b))) + \\ & \quad \log(f(B = b, C = c)/(f(B = b)f(C = c))) \end{aligned}$$



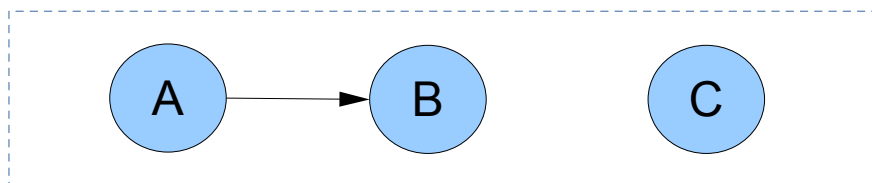
$$\begin{aligned} & \log f(A = a) + \log f(B = b) + \log f(C = c) + \\ & \quad \log(f(A = a, B = b)/(f(A = a)f(B = b))) + \\ & \quad \log(f(B = b, C = c)/(f(B = b)f(C = c))) \end{aligned}$$

Algorithm 2: Chow-Liu Trees

Short hand notation: $f(X = x) = \text{freq}(\{X = x\})$

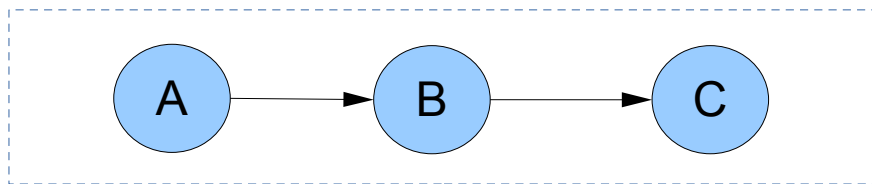
Intuitions

$$\log P(A = a, B = b, C = c) =$$

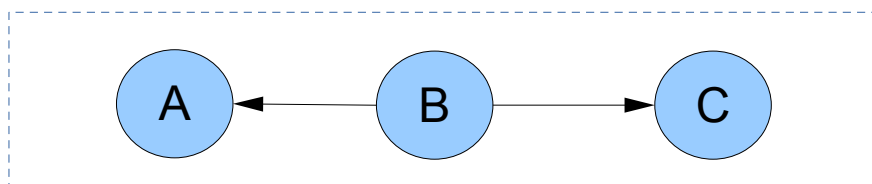


Part of the probability is
independent of the structure of the graph

$$\log f(A = a) + \log f(B = b) + \log f(C = c) + \log(f(A = a, B = b) / (f(A = a)f(B = b)))$$



$$\log f(A = a) + \log f(B = b) + \log f(C = c) + \log(f(A = a, B = b) / (f(A = a)f(B = b))) + \log(f(B = b, C = c) / (f(B = b)f(C = c)))$$



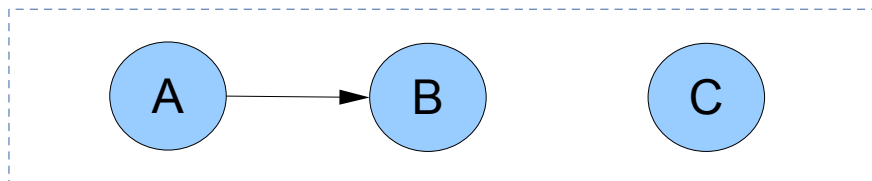
$$\log f(A = a) + \log f(B = b) + \log f(C = c) + \log(f(A = a, B = b) / (f(A = a)f(B = b))) + \log(f(B = b, C = c) / (f(B = b)f(C = c)))$$

Algorithm 2: Chow-Liu Trees

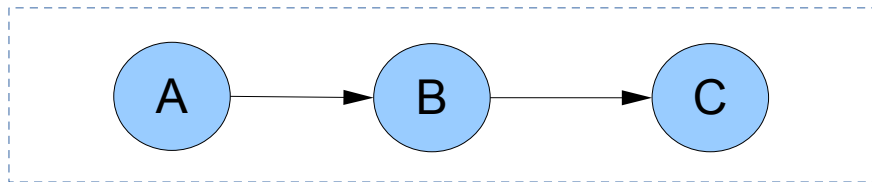
Short hand notation: $f(X = x) = \text{freq}(\{X = x\})$

Intuitions

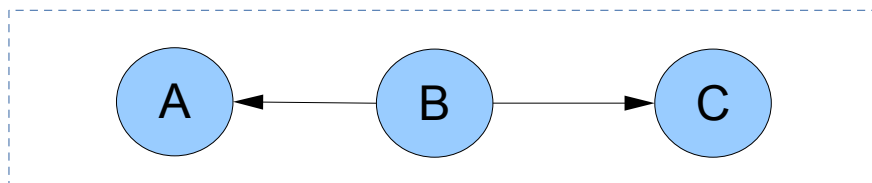
$$\log P(A = a, B = b, C = c) =$$



$$\log f(A = a) + \log f(B = b) + \log f(C = c) + \\ \log(f(A = a, B = b)/(f(A = a)f(B = b)))$$



$$\log f(A = a) + \log f(B = b) + \log f(C = c) + \\ \log(f(A = a, B = b)/(f(A = a)f(B = b))) + \\ \log(f(B = b, C = c)/(f(B = b)f(C = c)))$$



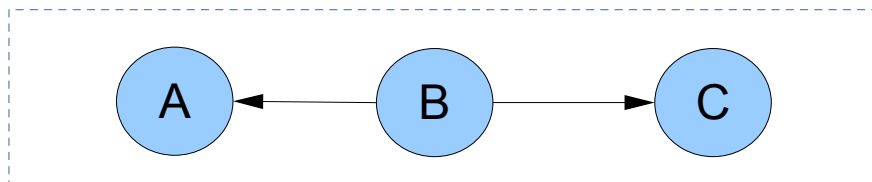
$$\log f(A = a) + \log f(B = b) + \log f(C = c) + \\ \log(f(A = a, B = b)/(f(A = a)f(B = b))) + \\ \log(f(B = b, C = c)/(f(B = b)f(C = c)))$$

Trees with the same selection of edges yield the same probability

Algorithm 2: Chow-Liu Trees

Short hand notation: $f(X = x) = \text{freq}(\{X = x\})$

Intuitions



$$\log P(A = a, B = b, C = c) =$$

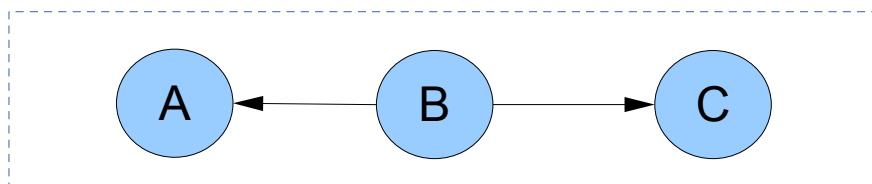
$$\begin{aligned} & \log f(A = a) + \log f(B = b) + \log f(C = c) + \\ & \log(f(A = a, B = b)/(f(A = a)f(B = b))) + \\ & \log(f(B = b, C = c)/(f(B = b)f(C = c))) \end{aligned}$$

$$\begin{aligned} \log P(\mathcal{D}) &= \sum_{(a,b,c) \in \mathcal{D}} \log P(A = a, B = b, C = c) = \sum_{(a,b,c) \in \mathcal{D}} \log f(A = a) + \log f(B = b) + \dots \\ &= |\mathcal{D}| \left(\sum_{a \in \{0,1\}} f(A = a) \log f(A = a) + \right. \\ & \quad \sum_{b \in \{0,1\}} f(B = b) \log f(B = b) + \\ & \quad \sum_{c \in \{0,1\}} f(C = c) \log f(C = c) + \\ & \quad \sum_{a,b \in \{0,1\}} f(A = a, B = b) \log(f(A = a, B = b)/(f(A = a)f(B = b))) + \\ & \quad \left. \sum_{b,c \in \{0,1\}} f(B = b, C = c) \log(f(B = b, C = c)/(f(B = b)f(C = c))) \right) \end{aligned}$$

Algorithm 2: Chow-Liu Trees

Intuitions

Short hand notation: $f(X = x) = \text{freq}(\{X = x\})$



$$\log P(A = a, B = b, C = c) =$$

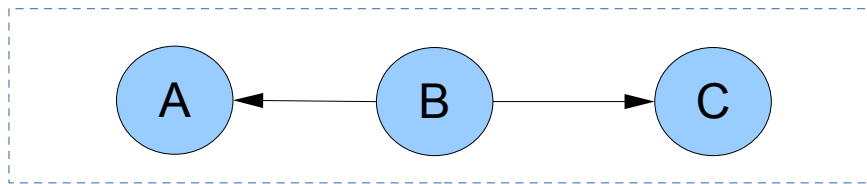
$$\begin{aligned} & \log f(A = a) + \log f(B = b) + \log f(C = c) + \\ & \log(f(A = a, B = b)/(f(A = a)f(B = b))) + \\ & \log(f(B = b, C = c)/(f(B = b)f(C = c))) \end{aligned}$$

$$\begin{aligned} \log P(\mathcal{D}) &= \sum_{(a,b,c) \in \mathcal{D}} \log P(A = a, B = b, C = c) = \sum_{(a,b,c) \in \mathcal{D}} \log f(A = a) + \log f(B = b) + \dots \\ &= |\mathcal{D}| \left(\sum_{a \in \{0,1\}} f(A = a) \log f(A = a) \right) + \quad \text{Entropy of } A \\ & \quad \sum_{b \in \{0,1\}} f(B = b) \log f(B = b) + \\ & \quad \sum_{c \in \{0,1\}} f(C = c) \log f(C = c) + \\ & \quad \sum_{a,b \in \{0,1\}} f(A = a, B = b) \log(f(A = a, B = b)/(f(A = a)f(B = b))) + \\ & \quad \sum_{b,c \in \{0,1\}} f(B = b, C = c) \log(f(B = b, C = c)/(f(B = b)f(C = c))) \end{aligned}$$

Algorithm 2: Chow-Liu Trees

Intuitions

Short hand notation: $f(X = x) = \text{freq}(\{X = x\})$



$$\log P(A = a, B = b, C = c) =$$

$$\begin{aligned} & \log f(A = a) + \log f(B = b) + \log f(C = c) + \\ & \log(f(A = a, B = b)/(f(A = a)f(B = b))) + \\ & \log(f(B = b, C = c)/(f(B = b)f(C = c))) \end{aligned}$$

$$\log P(\mathcal{D}) = \sum_{(a,b,c) \in \mathcal{D}} \log P(A = a, B = b, C = c) = \sum_{(a,b,c) \in \mathcal{D}} \log f(A = a) + \log f(B = b) + \dots$$

$$= |\mathcal{D}| \left(\sum_{a \in \{0,1\}} f(A = a) \log f(A = a) + \right.$$

$$\sum_{b \in \{0,1\}} f(B = b) \log f(B = b) +$$

$$\sum_{c \in \{0,1\}} f(C = c) \log f(C = c) +$$

Mutual Information between A and B: MI(A,B)

$$\sum_{a,b \in \{0,1\}} f(A = a, B = b) \log(f(A = a, B = b)/(f(A = a)f(B = b))) +$$

$$\sum_{b,c \in \{0,1\}} f(B = b, C = c) \log(f(B = b, C = c)/(f(B = b)f(C = c)))$$

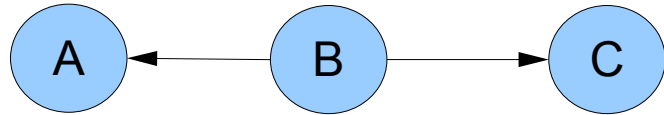
Algorithm 2: Chow-Liu Trees

Intuitions

Short hand notation: $f(X = x) = \text{freq}(\{X = x\})$

$$\log P(A = a, B = b, C = c) =$$

$$\begin{aligned} &\log f(A = a) + \log f(B = b) + \log f(C = c) + \\ &\quad \log(f(A = a, B = b)/(f(A = a)f(B = b))) + \\ &\quad \log(f(B = b, C = c)/(f(B = b)f(C = c))) \end{aligned}$$



$$\log P(\mathcal{D}) = |\mathcal{D}| \left(\sum_{V \in \text{nodes}(G)} \text{entropy}(V) + \sum_{(X,Y) \in \text{edges}(G)} MI(X,Y) \right)$$

Algorithm 2: Chow-Liu Trees

General idea: we wish to find a tree G that maximizes

$$\log P(\mathcal{D}) = |\mathcal{D}| \left(\sum_{V \in \text{nodes}(G)} \text{entropy}(V) + \sum_{(X,Y) \in \text{edges}(G)} MI(X,Y) \right)$$

which we can find by searching a tree G such that

$$\sum_{(X,Y) \in \text{edges}(G)} MI(X,Y)$$

is maximized

Algorithm 2: Chow-Liu Trees

- 1) Calculate for every pair of variables X and Y the $MI(X,Y)$ score, and put these on the edges of a graph MIG
- 2) Calculate a *maximum cost spanning tree* G of graph MIG , using any algorithm for calculating such spanning trees (Prim's algorithm, Kruskal's algorithm), e.g. using the following steps:
 - 1) Let G be a graph without edges and one node per variable
 - 2) As long as G is not a spanning tree of MIG :
 - 1) Select the edge (X,Y) from MIG that has the largest weight, is not in G yet and does not create a cycle in G
 - 2) Add (X,Y) to G
- 3) Pick an arbitrary node of the spanning tree G as the root, and point all edges away from this root
- 4) Fill in the CPTs of every node of G

Markov Networks: Learning Parameters

- Let's rephrase log-likelihood of the data:

$$\begin{aligned}\log P(\mathcal{D}) &= \sum_{(t,T) \in \mathcal{D}} \log P(T) = \sum_{(t,T) \in \mathcal{D}} \log \left(\frac{1}{Z} \exp \left(\sum_f w_f \cdot \text{match}(I_f, T) \right) \right) \\ &= -n \log Z + \sum_f w_f \cdot \sum_{(t,T) \in \mathcal{D}} \text{match}(I_f, T) \\ &= -n \log Z + \sum_f w_f \cdot \text{support}_{\mathcal{D}}(I_f)\end{aligned}$$

- Common solution: gradient ascent towards maximum log-likelihood parameters

$$\frac{\delta \log P(D)}{\delta w_f} = -n \frac{\delta \log Z}{\delta w_f} + \text{support}_{\mathcal{D}}(I_f)$$

Markov Networks: Learning Parameters

- How to calculate $\frac{\delta \log Z}{\delta w_f}$?

- Remember that

$$Z = \sum_{I \in D(V)} \exp \left(\sum_f w_f \cdot \text{match}(I_f, I) \right)$$

↙ All assignments to all variables

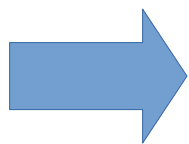
- Then

$$\frac{\delta \log Z}{\delta w_f} = \frac{1}{Z} \sum_{I \in D(V)} \frac{\delta \left(\exp \left(\sum_{f'} w_{f'} \cdot \text{match}(I_{f'}, I) \right) \right)}{\delta w_f}$$

Markov Networks: Learning Parameters

- How to calculate $\frac{\delta \log Z}{\delta w_f}$?

$$\begin{aligned}\frac{\delta \log Z}{\delta w_f} &= \frac{1}{Z} \sum_{I \in D(V)} \frac{\delta \left(\exp \left(\sum_{f'} w_{f'} \cdot \text{match}(I_{f'}, I) \right) \right)}{\delta w_f} \\ &= \frac{1}{Z} \sum_{I \in D(V)} \text{match}(I_f, I) \cdot \left(\exp \left(\sum_{f'} w_{f'} \cdot \text{match}(I_{f'}, I) \right) \right) \\ &= P(I_f)\end{aligned}$$



Can for instance
be calculated using (loopy) belief propagation,

But also Gibbs sampling can be used
(Hinton's “contrastive divergence”)

Markov Networks: Learning Parameters

- Algorithm:
 - 1) Initialize w_f randomly for all factors f
 - 2) Calculate $support(I_f)$ for all factors f
 - 3) Calculate $P(I_f)$ for all factors according to current weights (BP)
 - 4) Update $w_f \leftarrow w_f + \eta(support(I_f) - n \cdot P(I_f))$
for some learning rate η
 - 5) Goto 3, until some stopping criterion is met
- Intuition:
 - If the current weight predicts a lower expected support than the desired support, the weight for that factor should be increased

Markov Networks: Structure Learning

- The greedy search algorithm is much more complex than for Bayesian Networks, as parameter learning is much more complex
- Structure learning for Markov Networks is hence less common – structure is often assumed known
 - ◆ E.g., we assume every pair of variables is linked with a factor, and we learn the weights of all these factors

Bayesian Networks vs Markov Networks

	Advantages	Disadvantages
Bayesian Networks	• Interpretable • Easy to learn parameters • Easier to learn structure	• Can not be cyclic • Inference hard if not a trees
Markov Networks	• Somewhat interpretable • Can be cyclic	• Less interpretable • Harder to learn parameters • Harder to learn structure • Inference hard if not a tree

Message Passing

- Belief propagation is an example of an algorithm that performs *message passing* and can be used to solve well-defined reasoning problems exactly *on trees*
- Many machine learning algorithms rely on message passing, and can be linked to solving well-defined reasoning problems

However, the machine learning algorithms typically **never** solve a corresponding reasoning problem exactly.

- Next example: graph isomorphism and it links to graph neural networks (GNNs)

Example: Knowledge Graphs

Google Search for "Winterthur Zurich"

Wintherthur zurich

About 12,000,000 results (0.60 seconds)

Showing results for **Winterthur** zurich
Search instead for Wintherthur zurich

en.wikipedia.org > wiki > Winterthur

Winterthur - Wikipedia

Winterthur is a city in the canton of Zürich in northern Switzerland. With over 110,000 residents it is the country's sixth-largest city by population, and is the ...

Country: Switzerland

Lowest elevation (Kläranlage Hard): 393 ...

Highest elevation (Hulmen): 687 m (2,254 ft)

Elevation (Bahnhofplatz): 439 m (1,440 ft)

Battle of Winterthur · Winterthur, Delaware · Winterthur District · FC Winterthur

Top sights in Winterthur



Swiss Science Center Technorama
Science museum with hands-on exhibits



Castle Kyburg
11th-century castle with history museum



Oskar Reinhart collection Am
Art collection in a historic mansion



Kunst Museum Winterthur | Beim
Art museum, vincent van gogh, museum,...

Wikipedia page for "Winterthur"

Location of Winterthur

[show]



☒ Show map of Switzerland
☐ Show map of Canton of Zürich
☐ Show all

Coordinates: 47°29′56″N 8°43′43″E﻿ / ﻿

Country	Switzerland
Canton	Zürich
District	Winterthur
Government	
• Executive	<i>Stadtrat</i> with 7 members
• Mayor	<i>Stadtpräsident</i> (list) Michael Kunzle CVP/PDC (as of 2012)
• Parliament	<i>Grosser Gemeinderat</i> with 60 members
Area ^[1]	
• Total	68.07 km ² (26.28 sq mi)
Elevation (Bahnhofplatz)	439 m (1,440 ft)
Highest elevation (Hulmen)	687 m (2,254 ft)
Lowest elevation (Kläranlage Hard)	393 m (1,289 ft)
Population (2018-12-31) ^[2]	

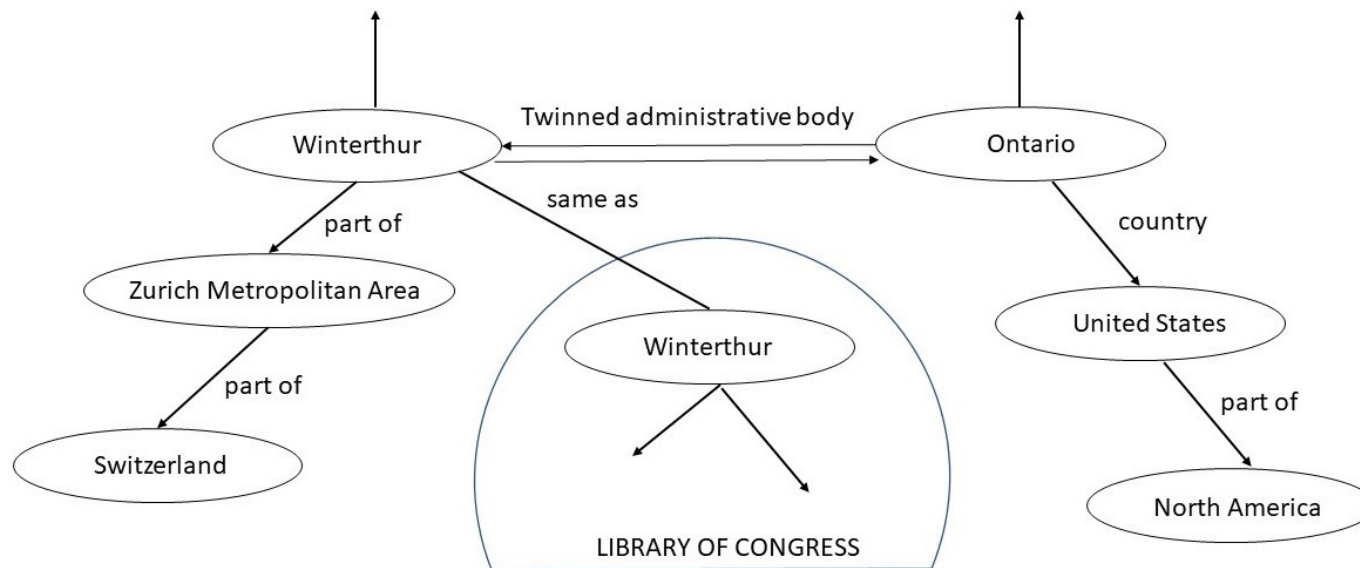
Knowledge graphs are a form of knowledge representation.
Google uses data from the Wikidata *knowledge graph* to provide additional information

Other well-known example: Yago (Yet Another Great Ontology)

Example: Knowledge Graphs

Most knowledge graphs consist of large sets of triplets
(source,target,label)

Wikidata: >90 million objects



Query languages have been developed over knowledge graphs,
which implement some form of logic

`part_of(x,"Zurich Metropolitan Area"),twinned(x,"Ontario")`

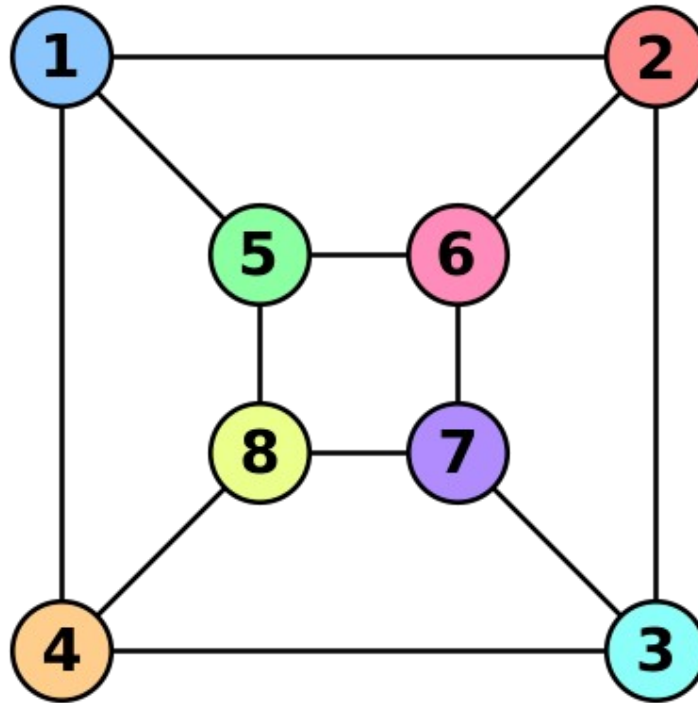
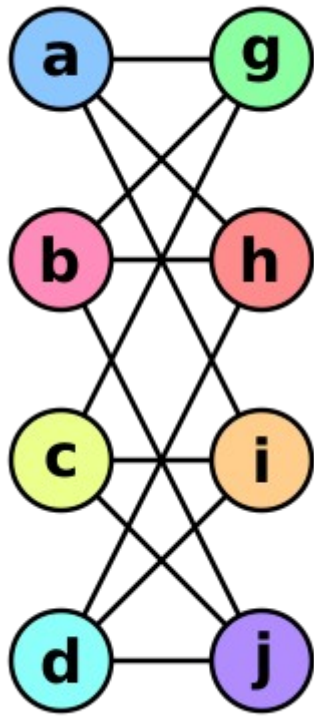
Example: Knowledge Graphs

- Query languages allow to extract information from knowledge graphs (reasoning: deduction)
- One can also be interested in learning to *classify nodes* in a knowledge graph (learning: induction)
 - ◆ In bioinformatics, can we learn which proteins are “active”?
 - ◆ In social networks, can we learn to predict the “alma mater” of a person?

Based on some information we already have about the nodes and their neighborhood

Graph Isomorphism

Two graphs G and H are isomorphic if there is a bijection f between the nodes such that any two vertices u and v of G are adjacent in G if and only if $f(u)$ and $f(v)$ are adjacent in H .



$$f(a) = 1$$

$$f(b) = 6$$

$$f(c) = 8$$

$$f(d) = 3$$

$$f(g) = 5$$

$$f(h) = 2$$

$$f(i) = 4$$

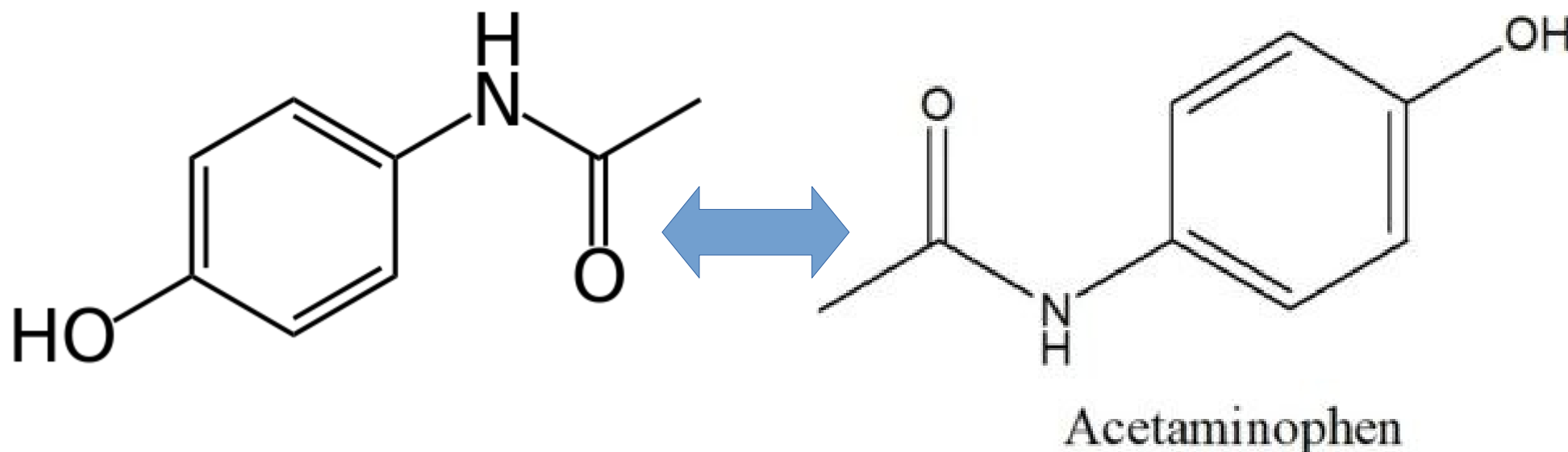
$$f(j) = 7$$

(illustration from Wikipedia)

Graph Isomorphism

Can be extended to node and edge labeled graphs

Can for instance be used to check equivalence between 2D structures of molecules



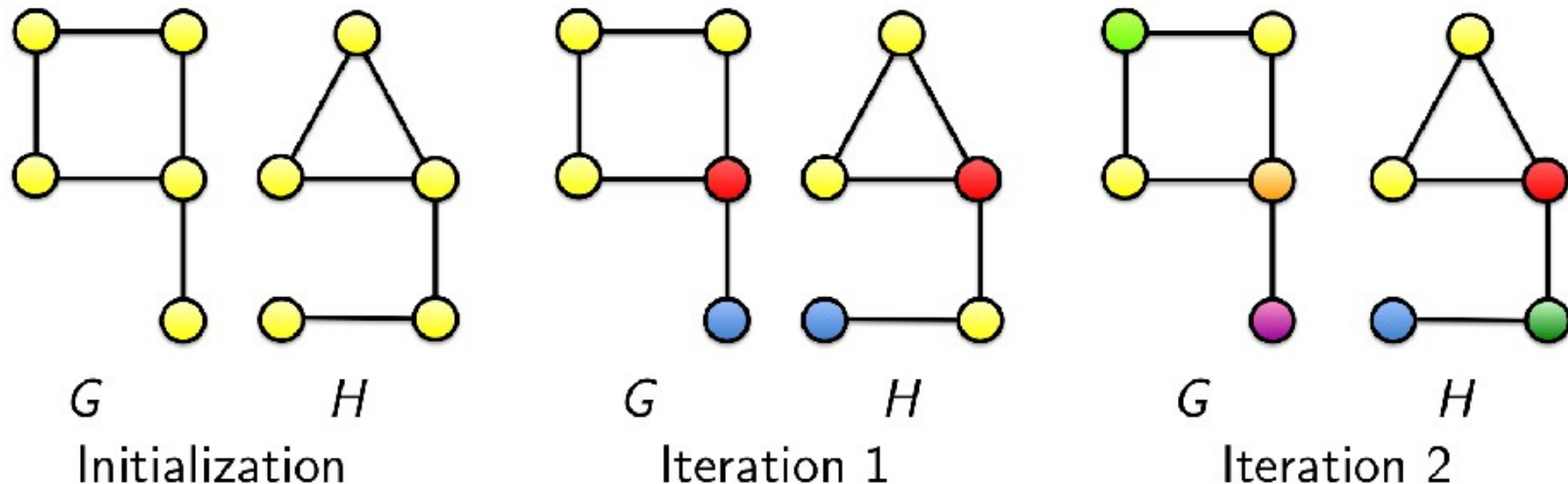
Can also be used to check equivalence of Datalog clauses
(~database queries)

Graph Isomorphism

- Not known to be NP-hard, but no polynomial algorithm known either
- Existing exact algorithms are hence **search algorithms** that search through the space of possible mappings between nodes
- To reduce the number of possible mappings, they use a form of propagation known as **color refinement** (or the Weisfeiler-Leman algorithm)

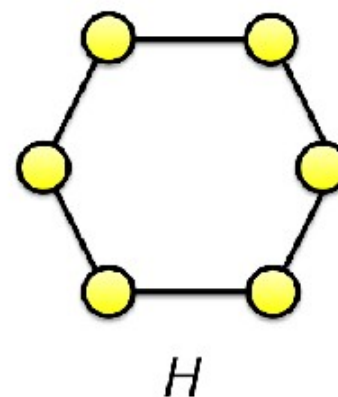
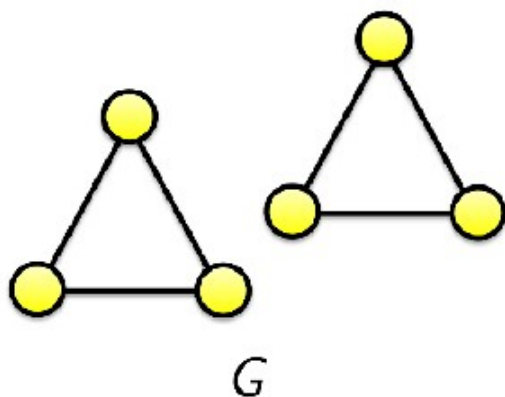
Color Refinement

- **Initial:** all vertexes have the same color
- **Iteration:** Further separation of colors based on color histograms of neighbors (“value of node updated based on messages from the neighbors”)
- Stop when no nodes with the same color can be separated further

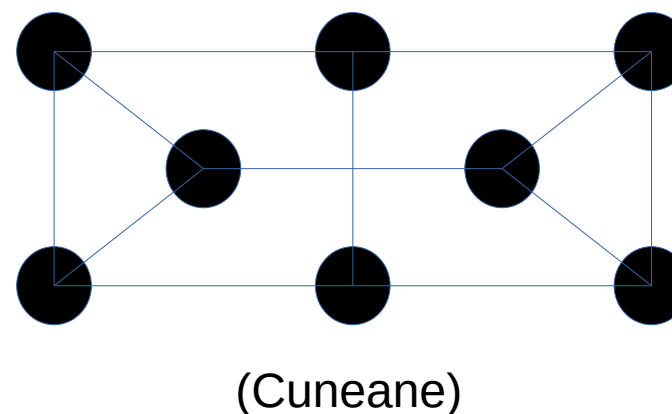
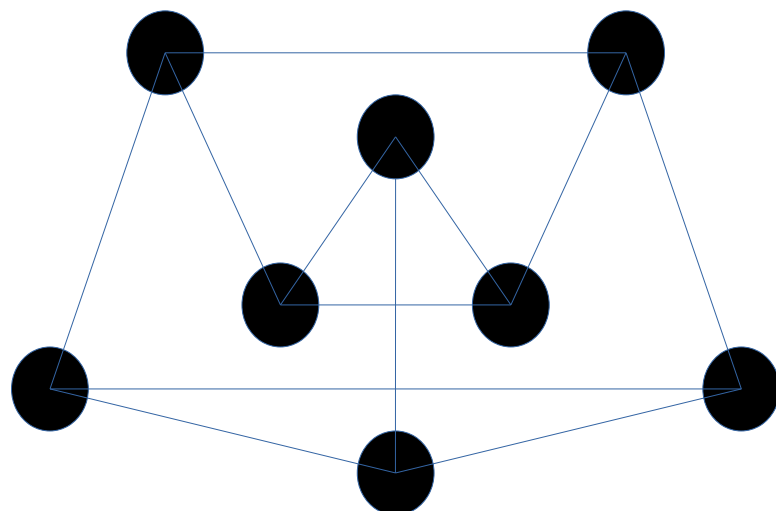


WL for Graph Isomorphism

- WL cannot distinguish these two *regular* graphs:



- Or between these two molecules:

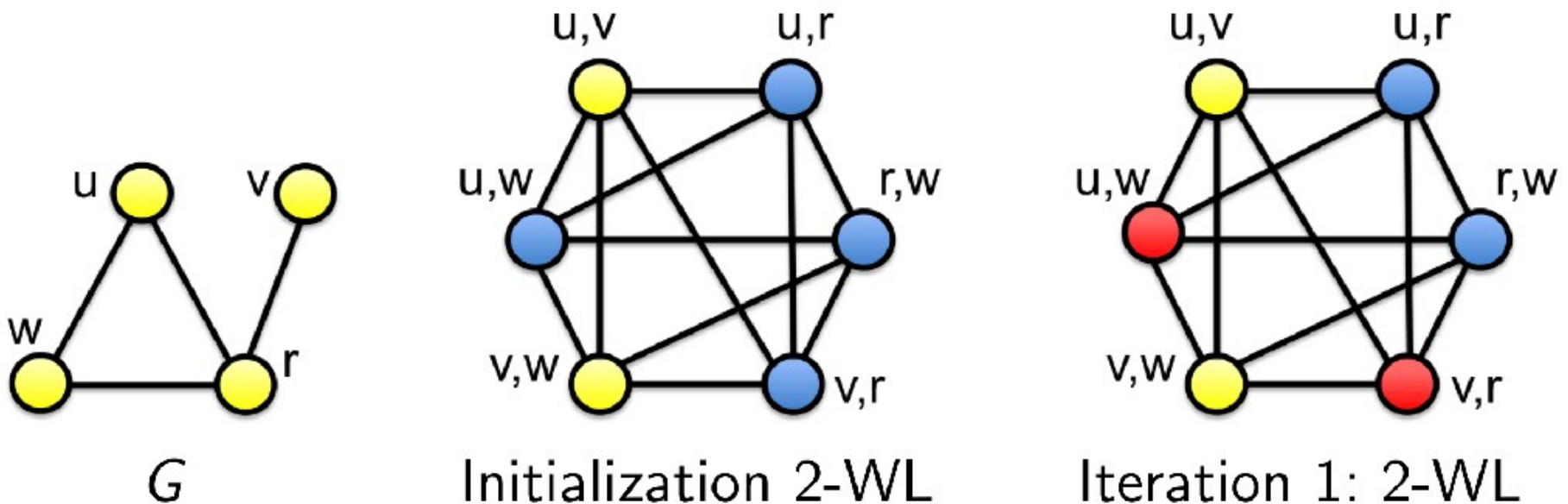


WL for Graph Isomorphism

- If two graph are isomorphic $\rightarrow$ WL will not find difference between the graphs
- If WL does not find a difference between graphs $\rightarrow$ it is still possible that the two graphs are not isomorphic
- Hence, additional search remains necessary to decide that two graphs are isomorphic

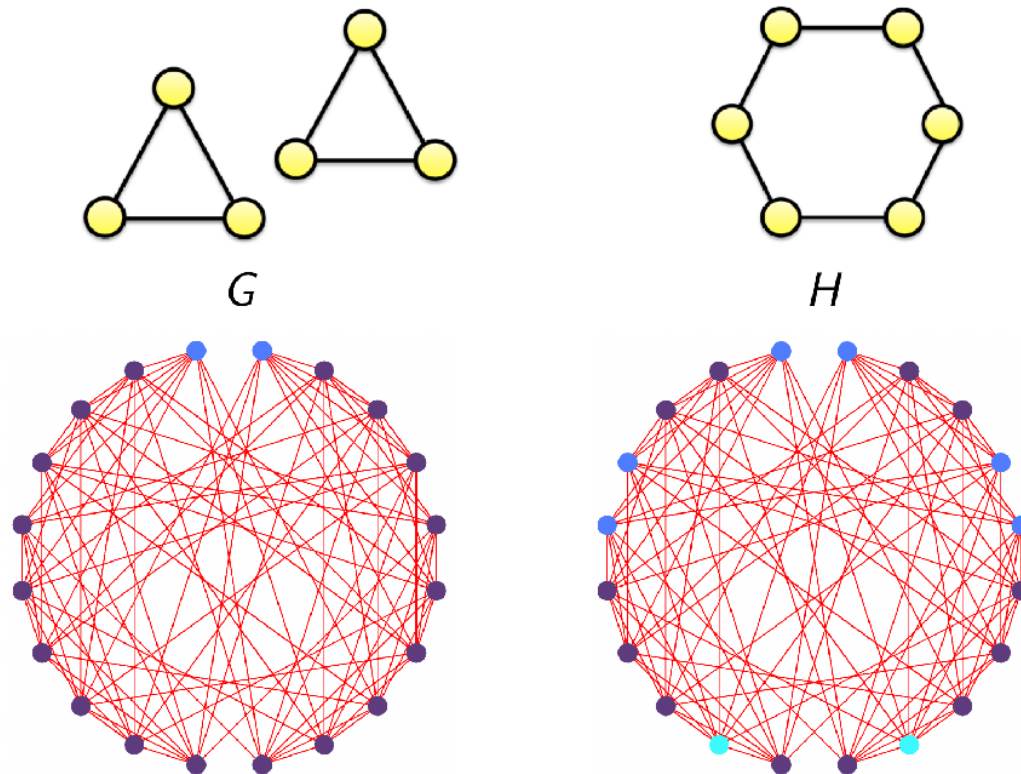
k -WL for Graph Isomorphism

- Two k -sets are neighbors if they differ in one element
- **Initial:** k -sets U, W get the same color if $G[U] \simeq G[W]$.
- **Iteration:** Two identically colored k -sets U and W get different colors if there exists a color c so that U and W have a different cardinality set of neighbors of color c .



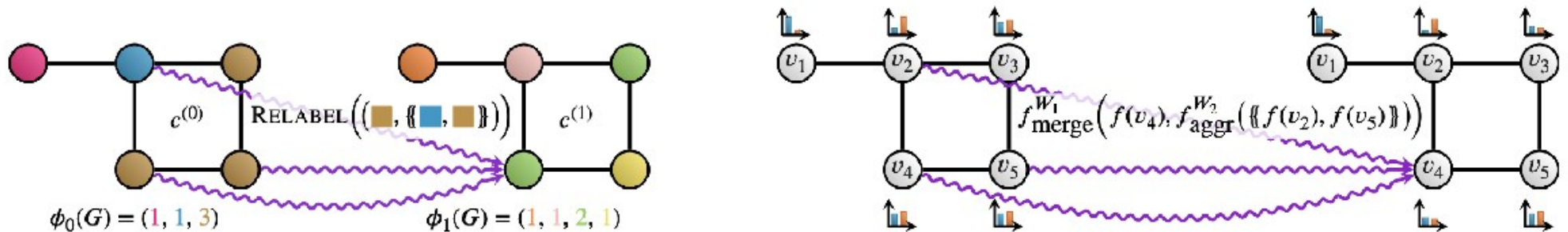
Power of k -WL

- For $k > 2$ k -WL is stronger than WL
- Exact for large k (for instance if $k = n$, by definition)
- For $k = 3$ regular graphs can be distinguished



Graph Neural Networks

- NNs for performing predictions for nodes in graphs



$$\mathbf{H}_v^t = f_{\text{merge}}(\mathbf{H}_v^{t-1}, f_{\text{aggr}}(\{\{\mathbf{H}_w^{t-1} \mid w \in N(v)\}\}))$$

- Where f_{aggr} aggregates embeddings of neighboring nodes and f_{merge} aggregates the neighborhood with the node's embedding to create a new embedding for the node
- Both functions are differentiable functions with parameters W_1, W_2
- Finally a multi-layer perceptron can be used to classify every node

Graph Neural Networks

- Two nodes that are equivalent according to WL, will also be classified equally by the GNN:

GNNs **cannot** detect graph isomorphism

- GNNs may even be able to distinguish **less nonisomorphic** nodes if the number of iterations is limited
- Results for k -WL however also apply to k -GNNs (but is computationally intensive, requires embeddings for larger graphs)

Graph Classification

- Several studies have linked the predictive power of GNNs to the expressive power of **logic**: what kind of classification rules in logic can be expressed in GNNs?
- Consider the following rule, which could be used to label a node with True or False:

$$\alpha(x) := \text{Red}(x) \wedge \exists y (E(x, y) \wedge \text{Blue}(y)) \wedge \exists z (E(x, z) \wedge \text{Green}(z)).$$

where $E(x, y)$ expresses that y is a neighbor of x

This rule can be expressed in a GNN.

- Consider this rule:

$$\beta(x) := \text{Red}(x) \wedge \exists y (\neg E(x, y) \wedge \exists z_1 \exists z_2 [E(y, z_1) \wedge E(y, z_2) \wedge z_1 \neq z_2 \wedge \text{Blue}(z_1) \wedge \text{Blue}(z_2)])$$

This rule cannot be expressed in a GNN; it requires some form of subgraph isomorphism, at arbitrary distance from x !

Graph Classification

- General result: formulas in *graded modal logic* can be expressed in GNNs.

These are formulas constructed using these building blocks:

$$\neg\phi(x), \quad \phi(x) \wedge \psi(x), \quad \exists^{\geq N} y (E(x, y) \wedge \phi(y))$$

where $\phi(x)$ is a predicate on an individual node

- An example:

$$\alpha(x) := \text{Red}(x) \wedge \exists y (E(x, y) \wedge \text{Blue}(y)) \wedge \exists z (E(x, z) \wedge \text{Green}(z)).$$

- Such formulas are often used in knowledge representation (for querying *knowledge graphs*)
- Hence trained GNNs have the ability to also represent common logical expressions used in this domain of knowledge representation

Conclusions

- In this first part, the focus was on graphs
- Many reasoning problems have been developed for graphs
 - ◆ Aggregation (calculating probabilities)
 - ◆ Maximization (maximum likelihood)
 - ◆ (Subgraph) isomorphism
- To solve these reasoning problems, well-known algorithms rely on forms of *message passing*.

However, *message passing* is by itself often not enough to solve these problems exactly

- Nevertheless, well-known machine learning algorithms now also rely on this message passing, losing exactness